

University of Illinois Urbana-Champaign

Project Report Phase II

CS513: Theory and Practice of Data Cleaning
Data Cleaning Project

Po Lin, Charlene Khun, Madalyn Killian (Team 38)
pohungl2@illinois.edu, ckhun2@illinois.edu, killian7@illinois.edu
Dataset Chosen: NYPL What's on the Menu Dataset

Brief Overview of Dataset and Project

Our group is using the NYPL What's on the Menu dataset. This dataset consists of a large collection of historical restaurant-menu records dating from the mid-1800s through the modern era, covering locations around the world. The dataset is split into four different tables: Menu, MenuPage, MenuItem, and Dish and tells us information such as what kinds of dishes are on menus, their corresponding prices as well as what when the dishes originated.

The group project focuses on using the NYPL dataset to categorize menus into relative price ranges, ranging from one to three dollar signs (in American dollars) within comparable historical time periods. Before any analysis can be made, the data must be cleaned because it contains missing or invalid prices, incomplete and impossible dates, inconsistent currencies, and unreliable dish names. We will use OpenRefine, Python, and SQL to profile, clean, validate, and prepare the records for reliable historical menu-price comparisons.

Project Use Case

U1 / Main: Our main use case is to categorize menu price ranges (\$, \$\$, \$\$\$) relative to the distribution of menu prices within comparable time periods. In order to achieve this analysis, we would need to restrict usable data to menus with usable dates and valid menu item prices. Data cleaning is necessary because:

- Menu item prices may be missing, malformed or inconsistent
- Menu dates and currencies may be incomplete or inconsistent
- Menus with too few priced items may produce misleading averages
- Dish names will need to be cleaned/normalized due to odd/vague attribute values
- Dish and menu items' prices would need to be inferred due to potentially missing prices between tables

We aim to get our data to a state where we are able to compare menus and their average dish prices grouped by similar time periods, as well as identify most/least expensive menus within those time periods.

1. Description of data cleaning performed

We cleaned the data in three layers using OpenRefine, Python/pandas, and SQLite. Each tool had a separate role: OpenRefine handled the initial profiling and scope reduction, Python applied repeatable value-level transformations, and SQLite checked the cleaned data against our relational constraints.

1.1 High-level cleaning steps

The work was divided across the team: Charlene handled OpenRefine profiling and scope filtering (section B), Po built the Python/pandas cleaning pipeline (section C), and Madalyn handled SQL validation (section E). Each pass writes a new output for the next step and never overwrites its input.

1.1.A Repository organized for provenance

`data/` is organized by source table, and within each table by processing stage, so any value in any output traces back to the file and row it came from. Every path below is a real directory in the submitted data set.

```
None
data/
├─ menu/ menu-page/ menu-item/ dish/
│   │   └─ raw/<Name>.csv
│   │   └─ interim/
│   │       │   └─ open-refine/<Name>_OR.csv
│   │       │   └─ func-<step>/
│   │       │       │   └─ <Name>_after_<step>.csv
│   │       │       │   └─ <Name>_changes_<step>.csv
│   │       │       │   └─ <Name>_summary_<step>.csv
│   │       │       └─ cleaned/<Name>_cleaned.csv
│   └─ reports
│       │   └─ sql-validation-results
│       │       │   └─ validation_fd_results.csv
│       │       │   └─ validation_ic_results.csv
│       │       │   └─ validation_ind_results.csv
│       │       └─ cleaning_log.csv
│   └─ final/cs513_team38.sqlite
```

one tree per source table
immutable input, never written to

OpenRefine export; input to the pipeline
one directory per cleaning function
full snapshot after that function ran
cell-level before/after, where applicable
rows affected per column and operation
load-ready for SQLite, provenance stripped

File showing rows that violate FD checks
File showing rows that violate IC checks
File showing rows that violate IND checks
Every Python/Pandas stage summary concatenated
SQL workflow outputs

Tracing a table's history. Each `func-<step>/` directory records one cleaning stage. Each cleaning function writes a complete snapshot of the table as it stood when that

function finished, so the effect of any single rule comes from diffing two adjacent directories. Each stage produces up to three levels of detail:

File	Answers
<code><Name>_after_<step>.csv</code>	What did the whole table look like after this rule?
<code><Name>_changes_<step>.csv</code>	Which individual cells changed, and from what to what?
<code><Name>_summary_<step>.csv</code>	How many rows did each operation affect, per column?

The code creates stage directories at runtime instead of requiring them in advance. Adding a cleaning rule only requires only to write an additional function; its directory appears on the next run and a deleted rule leaves no empty directory behind.

The number of stage directories differs per table, because each table runs only the rules its column types require. This is intentional and is what the W2 model (section 3.2) depicts:

Table	Stages	Chain
MenuPage	1	<code>id-numeric</code>
Menu	4	<code>id-numeric -> date-split -> currency-map -> drop-status</code>
Dish	4	<code>id-numeric -> dish-name -> price-decimal -> price-outlier</code>
MenuItem	4	<code>id-numeric -> price-decimal -> price-outlier -> omit-check</code>

Submitted files: 45 files. Only 8 of them are inputs: the four raw CSVs and the four OpenRefine exports. The other 37 files and all 18 of their directories are produced by the pipeline. The OpenRefine operation histories are in `workflow/open-refine/<Name>_OpenRefine_History.json`, alongside the YesWorkflow models in `workflow/yes-workflow/`, because both describe how the

data was produced rather than being data themselves. Additional details on these directories are covered in section 3.2.

Rebuilding the data set: With the eight input files in place, rebuilds every generated file:

```
Shell  
python -m src.run_pipeline
```

We verified this from an empty generated-data tree. We deleted all 37 generated files and all 18 generated directories, leaving only the eight inputs, and re-ran the pipeline. Every one of the 37 files was reproduced identically. Nothing in the generated data depends on run history, on a partially populated directory, or on the order in which stages were first written. This makes the `func-<step>/ snapshots` usable as evidence rather than as a record of one particular session. The rebuild is deterministic since each stage rewrites its own outputs and deletes any of its own files that a run no longer produces.

1.1.B Open Refine

Step 1 **Selecting only the columns needed for U1:**

In OpenRefine, we reduced each table to the attributes needed for the historical menu-price analysis. For `Dish.csv` we kept it limited to the following variables: `ID`, `name`, `lowest_price`, and `highest_price`. For `MenuItem.csv`, we kept the variables `id`, `menu_page_id`, `price`, `high_price`, and `dish_id`. Next for `MenuPage.csv`, we kept the `id` and `menu_id`. Lastly, for `Menu` we kept the `id`, `date`, `currency`, and `status`. Modifying the table this way helped us remove unrelated descriptive fields and made the datasets easier to inspect, process, and export. In addition, it also preserved the identifiers needed to connect `MenuItem` to `MenuPage`, `MenuPage` to `Menu`, and `MenuItem` to `Dish`. Having this reduction did not necessarily require us to calculate price ranges since unused columns would not change the results. However, this step was still useful because it reduced unnecessary complexity, lowered the amount of data carried into later stages, and made the OpenRefine workflow more focused on U1.

Step 2 **Standardizing data types:**

We converted the identifier fields (ID fields) in Dish.csv, MenuItem.csv, and MenuPage.csv to numeric values. We also converted the relevant price fields to numeric values as well. Menu dates were standardized as date-time values in a consistent ISO 8601 format, while the currency and status remained strings. This step was required for U1 because numeric prices are necessary to calculate menu averages and assign menus to the different dollar sign groups.

Consistent identifier types are also necessary for connecting records across the four tables. Furthermore, properly formatted dates allow menus to be grouped into comparable historical periods. Without standardized types, values could be treated as text, joins could fail, and calculations or chronological groupings could produce errors.

Step 3 **Handling blank Dish price values:**

In Dish.csv, records with blank values in both lowest_price and highest_price were excluded from the OpenRefine output. We only need the dish price information when a MenuItem.csv does not contain a usable price and we can use its corresponding dish prices as a backup. This step directly supports U1 because a dish with no recorded price (low or high) cannot provide a valid fallback price.

Keeping such rows could possibly skew the analysis and create the false impression that additional menu items had usable pricing information. Therefore, excluding these records ensures that the fallback table contains only dishes with at least one valid price that can be used to estimate a missing MenuItem price.

Step 4 **Retaining blank MenuItem prices for later fallback processing:**

Unlike Dish.csv, MenuItem.csv was not filtered in OpenRefine based on blank price or high_price values. As a result, the MenuItem OpenRefine export retained all 1,332,726 input rows, meaning that no MenuItem records were removed during this stage. The columns price, high_price, and dish_id were deliberately preserved because they are evaluated together in the later Python/Pandas workflow. A MenuItem with a blank direct price may still contain a dish_id that connects it to a Dish record with a usable lowest_price or highest_price, allowing its price to be recovered through the project's fallback logic.

Therefore, a blank MenuItem price alone was not considered sufficient justification for removing a row. Rather, the Python/Pandas stage applies a more complete omission rule and removes a MenuItem only when it has neither a usable positive price or high_price nor a dish_id that could support fallback processing. This later rule removed 237 records, reducing the MenuItem row count from 1,332,726 to 1,332,489. Preserving

all rows in OpenRefine therefore avoided prematurely discarding potentially recoverable pricing information while allowing the repeatable Python/Pandas logic to identify records that truly could not contribute to U1.

Step 5 Preserving and standardizing the MenuPage relationship table:

As mentioned in Step 1 and Step 2, for MenuPage.csv, we kept id and menu id and converted both fields to numeric values.

No null values were found in these selected columns. MenuPage essentially connects MenuItem.csv and Menu.csv, since a menu item references a menu page rather than directly referencing a menu. This step was required for U1 because menu-item prices must eventually be grouped by their corresponding menus. Even though MenuPage contains no pricing information itself, the preservation of id and menu id is essential for calculating each menu's average item price and assigning the menu to a relative price category.

Step 6 Restricting the Menu table to comparable currencies:

The data in Menu.csv was filtered to records where the currency value matched "Dollars" or "Cents." Records in other currencies and records with missing currency information were not included in the OpenRefine output. It is worth noting that although the filter allowed both "Dollars" and "Cents," the final OpenRefine output only contained records labeled "Dollars." The currency symbol column was also removed because it did not provide additional useful information for our U1 statement case. We also observed that whenever the currency value was null, the currency symbol value was null as well. This filtering was required for U1 because prices expressed in different currencies cannot be directly compared as though they share the same unit. The project could theoretically convert historical currencies, but doing so would require outside exchange rate and historical economic data. Therefore, such conversion was outside the scope of U1 and was identified as more appropriate for the "never enough" use case. Restricting the dataset to dollar-denominated menus therefore provides a more comparable population without introducing unverifiable assumptions.

Step 7 Filtering menus by completion status:

We kept only records whose menu status was "complete," which removed approximately 76 records. This step supports U1 because an incomplete menu may contain only part of its original list of items. Calculating an average price from an incomplete menu could produce a misrepresentation of that menu's price range / category (one to three dollar signs). For instance, a menu missing several expensive dishes could be classified as a less expensive restaurant than it actually was. Restricting our analysis to only complete menus improves consistency among the

menus being compared. Although completion status does not guarantee perfect data quality, it is a reasonable quality condition for deciding whether a menu contains enough information to support price classification.

Step 8 Cleaning and filtering menu dates:

The Menu date field was inspected and filtered in OpenRefine. Blank dates, uninterpretable as dates, and outliers were removed.

This included 237 blank dates, 23 non-date values, and one extreme outlier, resulting in a usable date range from 1850 through 2012. This step was required for U1 because the use case compares menu prices within similar historical periods. A menu without a valid date cannot be placed into a chronological group, and an impossible date could place the menu in the wrong period and distort / inflate comparisons.

Step 9 Profiling dish-name inconsistencies with clustering :

OpenRefine's clustering functions were used to inspect inconsistent dish names, including differences such as capitalization, punctuation, spelling inconsistencies. The profiling revealed examples such as multiple representations of broiled chicken, roast beef, coffee, and tea, as well as numeric or symbol-only dish names.

This step was not ultimately required for our U1 case because menus are classified using their numerical prices and dates rather than by grouping dishes according to standardized names. Merging historical dish names could possibly also introduce incorrect assumptions because many names are ambiguous or incomplete.

Nevertheless, this profiling was still useful because it documented an important limitation of the dataset. It also demonstrated OpenRefine's ability to detect duplicate textual categories. These findings helped justify our group's decision to focus U1 on numerical attributes rather than relying on inconsistent dish names.

Takeaway:

Overall, the OpenRefine work served as both an initial cleaning stage and a profiling stage. The steps involving prices, identifiers, dates, currencies, menu status, and table relationships were directly required to produce a comparable set of historical menus for U1. Column reduction was not strictly necessary for the final calculations, but it simplified the workflow and reduced the amount of irrelevant data carried into later stages.

Dish names served two distinct purposes. For one, OpenRefine clustering was used only to profile possible name variants, and our group intentionally declined to merge ambiguous historical dish names. Secondly, while the later Python/Pandas stage still

performed mechanical normalization (examples: standardizing capitalization, punctuation, separators, and whitespace), this mechanical normalization partly supported dish-level comparisons without assuming that similarly written names necessarily represented the same dish.

The OpenRefine stage also generated four exported files `Menu_OR.csv`, `MenuPage_OR.csv`, `MenuItem_OR.csv`, and `Dish_OR.csv`. These were used as the input files for the Python/Pandas cleaning pipeline.

Together, these steps documented data limitations, supported reproducibility, and provided the cleaned and scoped inputs needed for the later Python and SQL steps.

1.1.C Python/pandas type-based cleaning

We use thirteen column-type-specific functions across the four tables. We read every file as text (`dtype=str`) so a blank price stays distinguishable from a `0.00` price.

Pandas' type inference would otherwise treat two different problems into one.

Rule	Table.column	What it does
1	<code>all id, menu_id, menu_page_id, dish_id</code>	Coerce identifiers to integers, using a nullable integer type so a column containing absent values is not silently widened to floating point
2	<code>Menu.date</code>	Split the ISO 8601 timestamp into separate <code>year</code> , <code>month</code> , <code>day</code> columns, then drop the original string
3	<code>Menu.currency</code>	Map currency labels to ISO abbreviations (<code>Dollars</code> -> <code>USD</code>) through a lookup table
4	<code>Menu.status</code>	Drop the column, after confirming at run time that it holds a single value: "completed"
5	<code>Dish.name</code>	Normalize to uppercase ASCII alphanumerics plus apostrophe and comma: remove quotes and dashes, fold accents, expand <code>&</code> to <code>AND</code> , convert separators to spaces, delete remaining punctuation, collapse whitespace, mark non-Latin and symbol-only names <code>UNKNOWN</code>
6	<code>Dish.lowest_price,</code> <code>Dish.highest_price,</code> <code>MenuItem.price,</code> <code>MenuItem.high_price</code>	Render prices to two decimals so cent values are visible (<code>0.2</code> -> <code>0.20</code>), preserving blanks as blanks

7	Dish.lowest_price, Dish.highest_price, MenuItem.price, MenuItem.high_price	Winsorize the right tail: cap values above the 99.5th percentile at that percentile. Rows are never dropped and the left tail is untouched. We chose to abandon extreme values of the right tail due to the significant number of close-to-zero or zero values for the price columns.
8	MenuItem	Flag and remove rows with neither a dish_id nor any positive price, writing the flagged set, the reduced set, and the removed rows as three separate files

Provenance captured alongside every rule

Each stage writes three machine-readable provenance artifacts:

- a log column on every row, accumulating `<column>:<operation>` tags across the whole chain, For example, one row's history can read `date:date_split;date:column_dropped;currency:currency_mapped;status:column_dropped`
- a change log recording the before and after value of each modified cell with its source row number
- a stage summary counting rows affected per column and operation, concatenated into `data/reports/cleaning_log.csv`, which is the source for section 2.1.

`log`, `source_file`, and `source_row_num` stay in every stage snapshot but are stripped from the SQLite load-ready CSVs: because they describe the cleaning process rather than the cleaned data.

Rerunning the pipeline from the OpenRefine exports produces identical outputs. This is to preserve reproducibility of the cleaning process.

1.1.D SQL validation of the cleaned data

The four load-ready CSVs are loaded into SQLite as staging tables and checked by query. Validation is intentionally separate from cleaning: pandas rewrites values, SQL asks whether the result is internally consistent. Nothing in this layer modifies data; every check returns the rows that violate it, so an empty result means the constraint holds and a non-empty one is a countable finding rather than a silent repair.

Three families of check run against staging, each catching a different class of defect and each owned by a different member:

Type	Question it answers	File
IC	Is each value individually legal? Is a price non-negative, a year in range, a status drawn from its controlled vocabulary? (Owned by Charlene)	sql/03_validate_ic.sql
IND	Does every foreign key point at a row that exists? These are the U1 join path: menu -> page -> item -> dish (Owned by Madalyn)	sql/04_validate_ind.py
FD	Does one key ever carry two conflicting values? (Owned by Po)	sql/05_validate_fd.sql

The three are complementary rather than overlapping. An IC check reads one row at a time and cannot see that two rows contradict each other. An FD check compares rows sharing a key but cannot see that a key points nowhere. An IND check follows references but says nothing about the values at either end. A defect invisible to all three is not caught by this layer at all, which is why section 2.1's cell-level change counts and this section are both reported.

1.2 Rationale per step

U1 classifies menus into price tiers relative to other menus of the same period, so it needs three things to be trustworthy: the menu's date, the prices of its items, and a currency making those prices comparable. Each rule is justified against that, and three of the eight are only partly required.

Rule	Required for U1?	Why
Identifiers	Yes , indirectly	U1 needs average price <i>per menu</i> , which exists only after joining MenuItem -> MenuItemPage -> MenuItem and MenuItem -> Dish. A key in an inconsistent representation breaks that join.
Date split	Yes	"Comparable time period" is the grouping key of the entire use case. A menu with no usable year belongs to no group; an impossible year (0190, 1091, 2928) puts it in the wrong one
Currency mapping	Yes	The same number means different purchasing power in different currencies, so prices are not comparable until currency is explicit and uniform. Mapping to USD makes the restriction visible in the data rather than implicit in a filter
Drop status	No	Useful, not necessary. The column is constant after the preliminary OpenRefine filter, so it carries no information into staging, and dropping it keeps the staging schema honest about what actually varies.

Dish names	Partly	Not required to compute a menu's average price, which needs only prices and dates. It <i>is</i> required for the dish-level grouping that lets U1 compare like with like.
Price decimals	Yes	Averaging requires a consistent numeric representation, and cents are the unit prices are recorded in. Preserving blanks rather than defaulting them to zero is what keeps “missing” separate from “recorded as zero”; collapsing them would silently deflate every menu average as well as make price defaulting logic include incorrect values during calculation.
Outlier capping	Partly	Menu averages are sensitive to extremes, so an implausible price distorts the tier its menu lands in. In practice the effect is small, because the right tail turns out to be a currency artifact rather than transcription error — see below
Omit check	Yes	A row with neither a <code>dish_id</code> nor a positive price cannot be priced by any route, including the dish-level fallback, so it can never contribute to a menu average. This is the only row deletion the pipeline performs: 236 rows, 0.018%

On “Outlier Capping”, three decisions are worth stating because each rejects an obvious alternative:

- Winsorization replaces an extreme value with the threshold and keeps the row. Deleting would change how many priced items each menu has, which changes the average that U1 compares and would delete rows for being unusual rather than for being wrong.
- The left tail of a menu price distribution is real data: the median item is \$0.40 and a \$0.05 coffee is a genuine price. We tested a symmetric 3% trim and it moved the per-menu average by +8.7% for the cheapest quartile of menus and –32.0% for the priciest, compressing the very price range U1 exists to measure. Capping only the right tail at 0.5% moves the priciest quartile by –0.1% and the other three by 0.0%.
- We found only 28 (0.7% of the entire dataset) sit on dollar-denominated menus. The rule is actually doing less work than it appears to.

There were two rules that we deliberately did not incorporate into the Python/Pandas workflow:

- 445,974 MenuItem rows have a `dish_id` but no price of their own. Resolving those is the fallback logic of our plan, which needs Dish values alongside MenuItem and so belongs in SQL. Dropping them in pandas would have affected the pricing decision logic.
- Converting non-dollar menus to USD needs historical exchange rates, which are external data; our Phase-I argument for U1 rests on no external data being required. Non-dollar menus are excluded and the limitation stated rather than papered over.

2. Document & resolve remaining data quality issues

2.1 Summary table of changes

The three cleaning stages are reported separately because each one changes the data in a different way, and collapsing them would make it impossible to tell which stage is responsible for a given number. OpenRefine removes rows and columns from scope (stated in Section 1 of the report); Python rewrites cell values; SQL resolves and excludes rows that validation shows cannot be used. The stages chain, so each one's output row count is the next one's input.

2.1.A OpenRefine stage - scope reduction

The OpenRefine stage substantially reduced the datasets to the rows and columns needed for U1. Across the four tables, 30 unnecessary columns were removed: Menu was reduced from 20 to 4 columns, MenuPage from 7 to 2, MenuItem from 9 to 5, and Dish from 9 to 4. Row filtering affected Menu and Dish only. Dish.csv decreased from 423,397 to 394,297 rows after 29,100 records with both lowest_price and highest_price blank were removed. Menu.csv decreased from 17,545 to 5,236 rows, removing 12,309 records in total: 11,996 were removed by restricting currency to Dollars, 76 were removed by keeping only status = complete, 236 were removed because the date field was blank, and one record with the outlier year 2928 was excluded.

MenuPage retained all 66,937 rows, and MenuItem retained all 1,332,726 rows because blank item prices may still be recoverable through the Dish fallback relationship. In addition to these row and column changes, identifier and price columns were converted to numeric types and Menu dates were standardized to ISO date-time values; however, these type conversions changed the representation of retained cells rather than their underlying values, so separate per-cell modification counts were not recorded during the OpenRefine stage.

Row and column counts below are measured from the delivered *_OR.csv files:

Table	Raw rows	Export rows	Rows removed	Raw cols	Export cols
Menu	17,545	5,236	12,309	20	4
MenuPage	66,937	66,937	0	7	2

Menuitem	1,332,726	1,332,726	0	9	5
Dish	423,397	394,297	29,100	9	4

Following are the menu CSV filters applied and its remaining row count after each filter:

Filter	Menus remaining	Removed
raw	17,545	-
<code>currency = Dollars</code>	5,549	11,996
<code>status = complete</code>	5,473	76
<code>non-blank date</code>	5,237	236
year 2928 excluded	5,236	1

2.1.B Python/Pandas stage - value cleaning

Cells changed counts cells whose value the pipeline rewrote. Rows flagged counts rows where a condition was recorded without the value being altered; a missing or zero price is reported, never filled or removed. The following tables were extracted from the cleaning logs that the logic generated.

Table	Column	Cells changed	Rows flagged	Rows excluded
Menu	<code>currency</code>	5,236	0	0
	<code>date</code>	5,236 (dropped after split)	0	0
	<code>year, month, day</code>	15,708 (5,236 × 3, created)	0	0
	<code>status</code>	5,236 (column dropped)	0	0
	<code>id</code>	0	0	0
MenuPage	<code>id, menu_id</code>	0	0	0
Dish	<code>name</code>	383,289	647 marked UNKNOWN	0

	lowest_price	315,775 formatted · 767 capped	222,566 zero or negative	0
	highest_price	320,522 formatted · 882 capped	218,014 zero or negative	0
	id	0	0	0
Menuitem	price	508,661 formatted · 4,251 capped	445,916 missing · 367 zero or negative	0
	high_price	65,755 formatted · 459 capped	1,240,821 missing · 8 zero or negative	0
	dish_id	0	241 absent	0
	(row level)	NA	NA	237

Row counts through the pipeline. Only Menuitem loses rows, and only 237 of them:

Table	OpenRefine export	Load-ready output	Removed
Menu	5,236	5,236	0
MenuPage	66,937	66,937	0
Dish	394,297	394,297	0
Menuitem	1,332,726	1,332,489	237

Dish name normalization by operation. These overlap: one name can be uppercased and have accents folded, so they sum to more than the 383,289 names changed:

Operation	Names
uppercased	380,613
separators converted to spaces	67,294
whitespace collapsed	62,095
punctuation deleted	40,806
& expanded to AND	11,029
accents folded	9,418

marked UNKNOWN	647
typographic quotes normalized	142

Price outlier capping by column. Values above the 99.5th percentile were capped at that percentile. No row was removed:

Column	Threshold	Values capped
MenuItem.price	300.00	4,251
MenuItem.high_price	260.00	459
Dish.highest_price	79.79	882
Dish.lowest_price	50.00	767

2.1.C SQL stage - resolution and exclusion

In the SQL phase, the three cleaned data files were loaded into the staging schema using `src/validation_resolution.py` which calls and runs `sql/04_validate_ind.py` to begin. Before loading the data, the script checks whether the SQLite database file already exists and removes it if so, ensuring a clean state for repeated runs. The `sql/04_validate_ind.py` then calls the `sql/01_schema_staging.sql` file. SQLite reports foreign violations during the loading process itself and the IND validations are logged simultaneously. As each table is loaded with the cleaned data, any rows that violate a foreign key relationship are identified and removed before insertion, then written out to `validation_ind_results.csv`. After the IND results are saved, `sql/04_validate_ind.py` then calls the `sql/03_validate_ic.sql` and `sql/05_validate_fd.sql` files to record the FD and IC violations, logging them in the `validation_fd_results.csv` and `validation_ic_results.csv` files respectively.

2.2 IC-violation Report

Every check is stated as a denial constraint and returns the violating rows, so an empty result means the constraint holds. Each is run twice: once against D, staging loaded from the raw CSVs, then against D', staging loaded from the cleaned CSVs. This is because a violation count means nothing without the count it started from. Source files:

- validation_ic_results.csv
- validation_ind_results.csv
- Validation_fd_results.csv

2.2.A Inclusion dependencies (IND)

The `sql/04_validate_ind.sql` file contains a schema that defines four staging tables, `stg_menu`, `stg_dish`, `stg_menu_page`, and `stg_menu_item`, all of which are connected through primary key and foreign key relationships that enforce referential integrity between them. Below are the results of running `sql/04_validate_ind.sql`, where each rule is defined along with the total violation occurrences per rule.

Rule	Constraint	Violations in D
IND-1	<code>MenuPage.menu_id ⊆ Menu.id:</code> Every page must belong to a menu.	51366
IND-2	<code>MenuItem.menu_page_id ⊆ MenuPage.id:</code> Every item must be listed on a page.	498556
IND-3	<code>non-null MenuItem.dish_id ⊆ Dish.id:</code> Every non-null dish as a menu item must exist as a dish.	7516

In addition to these violations, it was independently logged that all of the cleaned menus and dishes were loaded with no issues, since these tables had no foreign keys to check and no dependencies. The results show that the cascade logic is working as intended, as filtering tables with dependencies causes orphaned children to follow. The dominant cause of the loss was in removing the violating menus.

2.2.B Functional dependencies (FD)

The `sql/05_validate_fd.sql` checks five functional dependencies against the staging tables. Below are the results of running `sql/05_validate_fd.sql`, where each rule is defined along with the total violation occurrences per rule.

Rule	Constraint	Violations in D
FD-1	Menu.id -> currency, date parts: A menu id shouldn't carry two different dates or currencies.	0
FD-2	MenuPage.id -> menu_id: A page shouldn't belong to two different menus.	0
FD-3	MenuItem.id -> menu_page_id, price, high_price, dish_id: An item id shouldn't describe two different line items.	0
FD-4	MenuItem.(menu_page_id, dish_id) -> price: The same dish on the same page shouldn't have two different prices.	5712
FD-5	Dish.id -> name, lowest_price, highest_price: A dish id shouldn't map to two different dictionary entries.	0

The results show that the cleaned data only contained FD-4 violations, all from `stg_menu_item`. The majority of the cases are flagged as 'price_and_blank', meaning that the same dish appeared twice on one page, one priced correctly and one left as blank. The remaining cases were 'conflicting_prices', which were instances of different listings of the same dish having two or more varying prices.

2.2.C Integrity and domain constraints (IC)

The `sql/03_validate_ic.sql` checks 17 integrity and domain constraints against the staging tables. Below are the results of running `sql/03_validate_ic.sql`, where each rule is defined along with the total violation occurrences per rule.

Rule	Constraint	Violations in D
IC-1	Menu.id (primary key) must not be NULL.	0
IC-2	Menu.currency must be 'USD'	0
IC-3	Menu.year cannot be NULL, non-integer, or outside of the range 1851-2012	0

IC-4	Menu.month cannot be cannot be NULL, non-integer, or outside 1-12	0
IC-5	Menu.day must be an integer between 1 and 31.	0
IC-6	MenuItem.id must not be NULL.	0
IC-7	MenuItem.menu_page_id must not be NULL.	0
IC-8	A non-NULL MenuItem.price must not equal zero.	130
IC-9	A non-NULL MenuItem.high_price must be greater than or equal to zero.	0
IC-10	MenuItem.dish_id must not be NULL.	0
IC-11	When MenuItem.high_price = 0, MenuItem.price must not be NULL.	0
IC-12	MenuPage . id must not be NULL.	0
IC-13	MenuPage.menu_id must not be NULL.	0
IC-14	Dish.id must not be NULL.	0
IC-15	Dish.lowest_price must not be NULL and must be between 0 and 1100.	0
IC-16	Dish.highest_price must not be NULL and must be between 0 and 3100.	0
IC-17	Dish.highest_price must be greater than or equal to Dish.lowest_price.	0

The results file contains only 130 violations, and every single one is an IC-8. The results show proof of the successful cleaning pipeline in previous steps. The one case that appeared showed that 130 menu items had the variable 'price' set to 0, when it should be set to NULL. 0 is meaningfully different from NULL in this case.

2.3 SQL Violation Resolution and Final Tables

After identifying the violations, final analysis and cleaning on the staging tables was completed. The file `validation_resolution.py` runs `06_clean_violations.sql` to apply fixes to the found violations from previous steps, then re-runs `03_validate_ic.sql` and `05_validate_fd.sql` to re-check all violations (IND violations do not need to be checked again as they are unfixable), and finally runs `sql/07_final_schema.sql` to build the tables ready for actual U1 use.

2.3.A IC-8 and FD-4

`06_clean_violations.sql` fixes the two violation types IC-8 and FD-4, as they were the only rules that were violated in the IC and FD checks.

IC-8 defines that all prices should be NULL or non-zero. This fix defines that if a row has a 0 price but `high_price` contains a real, non-zero value, that the value from `high_price` gets copied into price. If `high_price` and price are set to zero, the row is dropped. If the price is zero and `high_price` is null, price is set to NULL. The order is important here as only genuinely unusable rows are deleted from the table.

FD-4 defines that the same dish on the same page should not have two different prices. There are two disposition types, `conflicting_prices` and `price_and_blank`. To fix this every `menu_page_id` and `dish_id` group with one or more distinct prices was found and the average was calculated (for both `price` and `high_price`), then the row with the smallest id in the group is kept while all over conflicting rows get deleted. For duplicate rows with blank prices, the blank price row was dropped while the non-null price row was kept. The order is also important here as the rows that are deleted are the genuine NULL price cases.

2.3.B Building the final tables

After re-running `03_validate_ic.sql` and `05_validate_fd.sql` to re-check all violations and ensure that there are 0 violations logged in the validation csv's, the `07_final_schema.sql` is ran to create the two final tables meant for U1 use. There are two tables, `final_menu` and `final_item`. They are built as follows:

- `final_menu`: `menu_id`, `cleaned_year`, `currency_clean`
- `final_item`: `menu_item_id`, `menu_id`, `dish_name`, `clean_item_price`

`final_item` is built by joining `stg_menu_item`, `stg_menu_page`, and `stg_dish` in order to consolidate the related tables into one flat table where one dish name and price is directly attached to the menu it came from. This matches the foreign key dependencies listed in the original staging schemas.

3. Workflow model

W1 is the end-to-end project: raw CSVs in, validated final tables out. W2 (Section 3.2) is not a separate workflow but the expansion of Step 4 inside it, where W1 shows one cleaning box, W2 shows the thirteen functions that box contains.

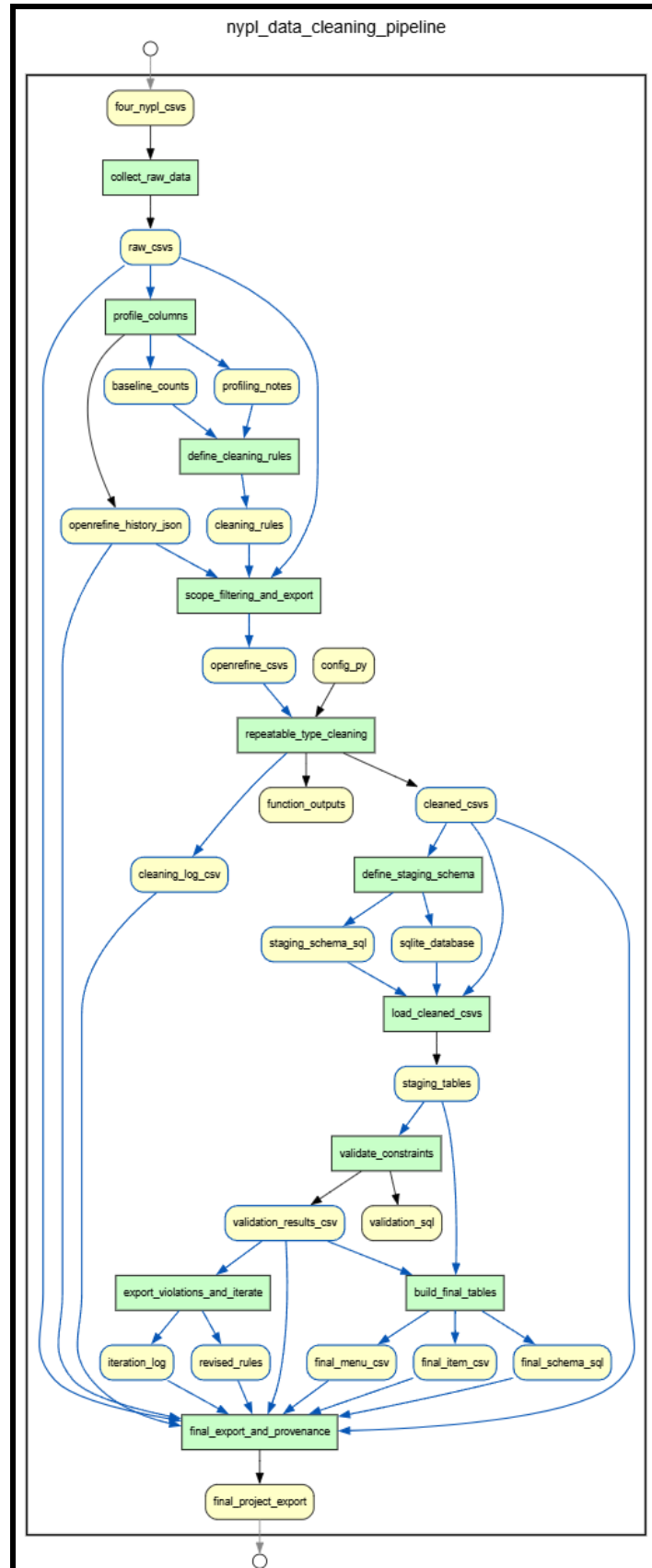
The organizing principle is that each step writes new files and never overwrites its input, so every arrow below is a file on disk rather than an in-memory handoff. That is what makes the workflow restartable from any step and auditable after the fact: any output can be traced back to the file and row it came from.

3.1 Outer workflow W1

An outer Yesworkflow diagram was generated from Yesworkflow. From the following website: <https://idaks.github.io/yw/>). See next page for Yesworkflow outer diagram.

The YesWorkflow outer workflow graph summarizes the full NYPL data-cleaning pipeline from raw data collection through final export and provenance. It begins with the four original CSV files, which are profiled in OpenRefine to produce baseline counts, profiling notes, and operation histories. Those results are then used to define cleaning rules and create scope-filtered OpenRefine exports.

The workflow continues with repeatable Python/Pandas cleaning, where standardized cleaned CSVs and a cleaning log are produced. SQLite is then used to define the staging schema, load the cleaned files into staging tables, and validate integrity constraints, inclusion dependencies, and functional dependencies. Validation results support both iteration and the construction of the final operational tables. The final step gathers the cleaned data, logs, validation outputs, SQL scripts, and provenance materials into one project export. Overall, the graph shows how each output becomes an input to a later stage, making the cleaning process transparent, traceable, and reproducible.



#	Step	Tool	Owner	Key inputs	Key outputs
1	Collect raw data, fix repository conventions	NA	Team	The four NYPL CSVs	data/<table>/raw/<Name>.csv
2	Profile columns	OpenRefine	Charlene	raw/<Name>.csv	Profiling notes, baseline counts, <Name>_OpenRefine_History.json
3	Define cleaning rules from the profile	NA	Charlene	Step 2 profile	Defined cleaning steps in this report
3b	Scope filtering and export	OpenRefine	Charlene	raw/<Name>.csv	interim/open-refine/<Name>_OR.csv
4	Repeatable type-based cleaning (W2)	Python/Pandas	Po	<Name>_OR.csv, src/config.py	interim/func-<step>/, interim/cleaned/<Name>_cleaned.csv, data/reports/cleaning_log.csv
5	Define staging schema	SQLite	Madalyn	Cleaned column shapes	sql/01_schema_staging.sql, data/cs513_team38.sqlite
6	Load cleaned CSVs into staging	SQLite	Madalyn	<Name>_cleaned.csv	stg_menu, stg_menu_page, stg_menu_item, stg_dish
7	Validate: IC, IND, FD	SQLite	Charlene / Madalyn / Po	Staging tables	sql/03-05_validate_*.sql, data/reports/validation_*_results.csv
8	Export violations, iterate	SQLite	Madalyn	Step 7 outputs	sql/03_validate_ic.sql, sql/05_validate_fd.sql, sql/06_clean_violations.sql
9	Build final operational tables	SQLite	Po	Validated staging	sql/07_final_schema.sql

10	Final export and provenance	NA	Team	Everything above	<code>data</code> directory, <code>src</code> directory (Python/Pandas), <code>sql</code> directory, report PDF, <code>workflow</code> directory (YW graphics/diagrams)
----	-----------------------------	----	------	------------------	---

Tool choices

Tool	Used for	Why this tool
OpenRefine	Steps 2, 3b: profiling and scope filtering	Yesworkflow (from https://idaks.github.io/yw/) was to generate a workflow model from each Open Refine history file (<code><Name>_OpenRefine_History.json</code>). It shows the sequence of profiling, filtering, transformation, and export operations performed on the data. This provides a visual and reproducible record of the OpenRefine process which makes it easier to verify how each exported file was produced and trace the workflow back to the recorded cleaning steps.
Python/Pandas	Step 4: type-based cleaning	The cleaning is per-column, rule-based, and has to run identically on 1.3M rows every time. Pandas gives vectorized column operations and, critically, explicit control over type coercion. It is also the only stage where a change log can be produced as a byproduct of the transformation rather than reconstructed afterwards.
SQLite	Steps 5–9: staging, validation, final tables	N/A. We did not use any workflows for the SQL code.
YesWorkflow	W2 model	The annotations live in the pipeline source, so the graph is generated from the code that runs and cannot drift from it. See section 3.2.

3.2 Inner cleaning workflow W2

W2 models the Python/pandas cleaning in detail.

The pipeline source carries YesWorkflow annotations, so the graph is generated from the code that runs rather than drawn separately. This section was done in collaboration with Claude (AI tool), which scans our Python/Pandas script holistically, defines functions and YesWorkflow annotations based on the functions that were created. Our involvement was to review and iterate on the proposed code until the desired output is achieved. (See Appendix A).

3.2.A YesWorkflow Open Refine

The YesWorkflow models document the OpenRefine cleaning performed on the Dish, Menu, MenuItem, and MenuPage tables. These workflow files were generated from the OpenRefine operation histories and represent each cleaning action as a separate computational step with clearly identified inputs and outputs. Across the four tables,

unnecessary columns were removed to retain only attributes relevant to the project's analysis and database relationships. For example, descriptive and administrative fields were removed from Dish and Menu, coordinate and timestamp fields were removed from MenuItem, and image and page-dimension fields were removed from MenuPage. The workflows also show that key identifiers, foreign keys, and price fields were converted to numeric values. These transformations help establish consistent data types and make the fields suitable for comparisons, integrity-constraint validation, and relational joins.

The workflows also capture table-specific cleaning decisions. In Dish, the lowest and highest price columns were converted to numbers, and a temporary dish_price_status column was created to identify records without usable price information before those rows were removed. In Menu, several row-filtering operations were applied to the identifier, currency, status, and date fields. The remaining date values were converted to OpenRefine date objects and then standardized into the yyyy-MM-dd'T'HH:mm:ss'Z' format. MenuItem standardized its primary key, foreign keys, and price columns as numeric values, while MenuPage converted both id and menu_id to numbers. By representing these actions in YesWorkflow, the project provides a reproducible provenance record showing how each original column was transformed into its cleaned output and how the final tables were prepared for later validation and analysis.

To see actual Open Refine Yesworkflow graphics, see appendix B.

3.2.B. YesWorkflow Python

23 annotated blocks across the frame file and five pipeline modules. Three annotation kinds carry the model:

- `@in / @out` name the data flowing between stages, each bound to a real file path with `@uri`. Adjacent stages connect because one's `@out` name matches the next one's `@in`.
- `@param` marks the configuration values a stage consumes: the currency map, the plausible year range, the dish-name character rules. These render distinctly from data, so the graph shows which rules are driven by which constants.
- `@desc` names the files each stage writes, so a node in the graph points at the artifact it produced.

MenuPage runs one stage; Menu runs four. The chains never join, because nothing in this pipeline joins across tables, this step happens in SQL. The asymmetry (described in 1.1.A) illustrates that each table is processed according to the column types it actually has, not put through a uniform sequence.

The top-level graph (Figure B.1) shows the four chains (one for each data file) as sub-workflow nodes, which is separate from per-data-file graph (Figures B.2-B.5) that expands each one's stages (See Appendix B or the workflow directory as part of the submission):

File	Contents	Nodes / edges
<code>workflow/W2_frame.yw</code>	Outer frame: opens the workflow, declares its external ports	NA
<code>workflow/W2.gv</code>	Top level: four table chains + the cleaning-log rollup	21 / 24
<code>workflow/W2_clean_menu.gv</code>	Menu's four stages	16 / 18
<code>workflow/W2_clean_dish.gv</code>	Dish's four stages	16 / 23
<code>workflow/W2_clean_menu_item.gv</code>	Menuitem's four stages, including the omit split	17 / 21
<code>workflow/W2_clean_menu_page.gv</code>	MenuPage's single stage	10 / 8

Each is rendered to PNG beside its `.gv` source. The top level uses a left-to-right layout; the per-table graphs use top-to-bottom. Annotation correctness was checked while the annotations were being written, using a linter Claude wrote for the purpose (`src/lint_workflow.py`). YesWorkflow exits successfully on several mistakes that silently produce a wrong graph, so the linter verifies that blocks are balanced, that every input has a producing output, that every parameter resolves to a real constant in `src/config.py`, that every file reference exists, and that none of YesWorkflow's keywords appears in ordinary prose. It is a development aid, not part of reproducing the data set.

The top-level view is four parallel tracks for each source data file, each running from its OpenRefine export through its cleaning block to its load-ready CSV, with the four outputs converging on the cleaning-log rollup and Menuitem carrying a second output for its omitted rows. The per-file views expand each block: Menu's shows five sequential stages with `NULL_LIKE_VALUES`, `MIN_PLAUSIBLE_YEAR`, `MAX_PLAUSIBLE_YEAR`, `DATE_PART_COLUMNS`, `CURRENCY_MAP`, and `PROVENANCE_COLUMNS` attached as parameter nodes to the specific stages that consume them; MenuPage's shows one.

4. Conclusions & summary

Cleaning ran as three stages in three tools, and the division was not arbitrary: OpenRefine reduced scope, Python rewrote values, and SQLite validated relationships. Each stage does something the other two cannot. OpenRefine's faceting makes a problem visible faster than writing code to look for it, but its import silently coerces types. Python can hold the blank-versus-zero distinction that the whole price analysis rests on, but it cannot see across tables. SQLite can follow a foreign key, but it never sees the raw string that was cleaned away. Conclusions are drawn per stage below for that reason.

The most important finding is not a dirtiness problem at all. Of 17,545 menus, 11,996 have a blank `currency`, and those menus carry 361,600 transcribed items with 473 prices between them. Currency is blank because nothing on the menu was ever price-transcribed, not because a label was lost. No amount of cleaning recovers them. Combined with the currency, status and date filters, U1's usable population is 5,236 menus, 29.8% of the file. That is a *completeness* ceiling inherited from the source, and it limits every conclusion the project can reach. We would rather state it plainly than present a cleaned dataset that implies more coverage than it has.

4.A OpenRefine

The OpenRefine profiling stage changed the cleaning plan by showing that the project's most useful evidence came from numerical prices, valid dates, comparable currencies, and reliable table identifiers rather than from dish names. Profiling revealed large amounts of menus with unusable currencies or dates, Dish.csv records with no fallback price, and MenuItem.csv records whose missing direct prices might still be recovered through `dish_id`. As a result, we narrowed U1 to dollar-denominated, complete menus with usable dates, removed Dish records that could not provide any price fallback, and preserved all 1,332,726 MenuItem rows for later evaluation. This is because a blank item price alone did not mean the price was unrecoverable. OpenRefine also helped rule out historical currency conversion because it would require external exchange-rate data, showing that aggressive dish-name merging was not dependable enough to support the main use case.

OpenRefine was especially useful for visually inspecting facets, identifying blank and unusual values, examining date outliers, comparing category frequencies, reducing columns, and filtering the data to the scope required for U1. Its clustering tools exposed differences in capitalization, punctuation, spelling, and formatting among dish names. This included factors of variants of foods such as coffee, tea, roast beef, and broiled

chicken. However, clustering was limited by the size and ambiguity of the Dish table, which contained approximately 423,000 records and many incomplete, numeric, symbol-only, or historically distinct names. Similar-looking names could not safely be assumed to represent the same dish, so automatic merges were declined to avoid introducing unsupported relationships. Rather, clustering was used for profiling, while later Python processing performed mechanical normalization such as standardizing capitalization, punctuation, separators, accents, and existing whitespaces.

An important lesson from this stage was that detecting a suspicious or missing value does not always justify immediately correcting or deleting its row. Decisions had to consider how the four tables would later be joined and whether missing information could be recovered through another table.

OpenRefine also had limitations. First, its import process could coerce data types. Secondly it could not apply the project's cross-table price-fallback logic. Lastly it did not provide the same repeatable cell-level provenance and relational validation later produced by Python and SQLite.

What remains unresolved includes the true equivalence of ambiguous dish names, prices missing from both Menuitem and Dish, menus excluded because their currencies or dates cannot be recovered, and non-dollar prices that cannot be converted without outside historical data. These unresolved issues represent limitations of the source data rather than problems that OpenRefine alone could safely repair.

4.B Python/Pandas

What the stage produced

Thirteen column-type-specific functions across four tables, running end to end in about 33 seconds and reproducing byte-identical output from the same inputs. The substantive changes: 6,359 extreme prices capped across the four price columns, every date split into year/month/day, currency mapped to a single ISO code, and 236 rows (0.018%) removed from Menuitem.

Four decisions carried most of the weight:

- **Read everything as text:** Letting pandas infer types would have merged a blank price and a 0.00 price into the same value, collapsing two different data-quality problems into one and silently deflating every menu average. This one line of configuration is the reason the rest of the analysis is trustworthy, and it is invisible in the output.

- **Flag instead of drop:** Every rule records what it found and keeps the row. 445,974 MenuItem rows have a `dish_id` but no price of their own; dropping them would have removed a third of the table and pre-empted a decision this stage cannot make, because resolving them needs Dish joined to MenuItem. Deferring that to SQL kept the option open.
- **Cap outliers instead of deletion:** We were asked to implement a symmetric 3% trim and measured its effect on the U1 metric before adopting it: it moved the per-menu average by +8.7% for the cheapest quartile and -32.0% for the priciest, compressing exactly the price range U1 exists to measure. Right-tail winsorization at 0.5% moves the priciest quartile by -0.1%.
- **Provenance Integration:** Each cleaner returns the operations it applied alongside the value, so the `log` column, the change logs and the summary counts fall out of the transformation instead of being reconstructed afterwards. Every number in section 2.1.B is generated by the pipeline rather than compiled by hand.

What we chose not to pursue in this stage:

- **Fallback pricing:** which was explained in our original Python/Pandas cleaning proposal, needs a cross-table join and belongs in SQL. Duplicate dish entries are reported, not merged: consolidating them would rewrite the identifiers that MenuItem rows depend on, which is a schema migration rather than a cleaning rule.

4.C SQLite

The staging and validation layer confirmed that the earlier cleaning steps were largely successful. This was seen immediately as once the tables were successfully loaded into the SQLite database with the properly enforced primary and foreign keys, almost none of the documented integrity constraints and functional dependencies were violated. It was shown that every cleaned table's fields were mostly populated, all dates and currencies fell within the expected ranges, and price ranges were mostly valid. As stated previously, the main validations that were caught during this process were the inclusion dependency violations (foreign keys). There were options to attempt to fix these violations, such as attempting to re-parent the orphans by making assumptions, but the approach that would give the most accurate results included dropping the violations given the strict requirements of the foreign keys.

4.D Plan vs. actual

The following describe deviations from our Phase I proposal that arose during our actual cleaning step in Phase II.

Planned	What happened	Why
Step 2 completes before Step 4 begins	Ran in parallel, with Python and SQL working from a preliminary OpenRefine export	Serial execution left two members idle. The cost (rerunning Step 4 if the export changes) is low because the pipeline is deterministic and reproduces byte-identical output
<code>Menu.currency_symbol</code> cleaned alongside <code>currency</code>	Dropped from scope	Blank on exactly the same rows as <code>currency</code> , so it carries no independent signal. This makes the planned "compare label against symbol" check a documented no-op rather than a skipped step
Section 4.6 of the plan: fallback pricing implemented in pandas	Moved to SQL	It needs MenuItem joined to Dish, which is a relational operation. Implementing it in pandas would have duplicated the join
None	Outlier detection and exclusion from the dataset	Upon profiling, we noticed that there were outlier values that would have significant effects on the relative average cost per menu. We included this cleaning mechanism to prevent certain menus from being disproportionately expensive by effect of outlier values.
MySQL as the database	SQLite	Phase-I named MySQL while the Phase-II task division assumed SQLite; SQLite needs no server and the database file ships with the deliverable

4.D.A Open Refine Deviations

The delivered OpenRefine exports generally match the filtering steps described in Section 1.1 B. The Menu export was reduced from 17,545 to 5,236 rows after restricting the data to dollar-denominated menus, retaining only complete menus, removing records with blank dates, and excluding the year-2928 outlier. Although the currency filter was configured to accept both "Dollars" and "Cents," the final export contained only

“Dollars,” so this is a documented outcome of the source data rather than an unexplained deviation. Dish was reduced from 423,397 to 394,297 rows by removing 29,100 records where both price fields were blank, while MenuPage and MenuItem retained all rows as intended because their relationship and fallback information was still needed in later stages. Therefore, the delivered exports are consistent with the stated OpenRefine scope filters, with the only noteworthy deviation being that no “Cents” records remained in the exported Menu data.

4.D.B SQL Deviations

The delivered SQLite tables were less detailed than the original schemas we defined in Phase I. As the final cleaned tables were created, it became clear that the extra fields were not needed to support the goals of U1 and were mostly there for logging and cleaning purposes, example being the `price_source` field as this would only be useful for further cleaning. In addition, the SQL cleaning was expected to have multiple iterations in order to reach a fully cleaned state, but this was unnecessary. In practice the amount of IC and FD violations that occurred were small and fixable that a single pass of cleaning in addition to another pass of violation checks was sufficient to reach a state of zero violations. This process was much more simple and linear than anticipated.

4.E Contributions

A detailed description of who was responsible for what specific tasks is located in the outer workflow table W1 in section 3.1 (pages 22-23 of the report). The work was distributed evenly and equally across the team, with each member also contributing to sections outside of their primary responsibilities whenever necessary and to validate each other's work throughout the process.

Appendix A - Tool use and AI assistance

Our Phase-I plan anticipated "AI-assisted code generation where appropriate and permitted by course guidelines" and listed ChatGPT among the S2 profiling tools, so this appendix documents AI use during the cleaning work itself. We've implemented a directive in Claude during code generation to document all interactions separately in a log file, detailing all human-in-the-loop interactions as well as the decisions we've made throughout the process.

A.1 AI assistance during the cleaning work

Tool: Claude Code (Anthropic Claude Opus 5), run locally against the repository.

Used across Python/Pandas logic definition step: sharing our preliminary OpenRefine profiling results, reorganizing the `data/` tree for provenance tracking, writing the pipeline modules based on our cleaning requirements (defined in step 1.1.C), generating the W2 workflow model and diagrams, and rebuilding the whole data set from its inputs to verify reproducibility.

Cleaning decisions were made by us; the tool produced analysis and code against those decisions. Where it disagreed with a decision it said so, and three of those objections changed the plan (the currency-rescue idea in Section A.1.8, the outlier method in Section A.1.9, and the pre-created directories in Section A.1.2 seen below).

Tool	Used for	How the output was verified
Claude Code	Sharing our profiling results from OpenRefine over the four raw CSVs: per-column blank/zero/non-numeric counts, year-range tails, currency cross-tabs, dish-name character census	Counts cross-checked against the preliminary OpenRefine facets. Three disagreements found and reconciled: <code>currency</code> distinct values (43 vs. our 39), <code>status = under review</code> (174 vs. our 167), <code>MenuItem.dish_id</code> blanks (241 vs. our 0)
Claude Code	Reorganizing <code>data/</code> into per-table <code>raw/interim/final</code> trees; moving 8 CSVs; rewriting <code>.gitignore</code>	Move script ran preflight assertions (every source present, no destination pre-existing) and compared byte sizes before and after each move; <code>git check-ignore</code> confirmed the new rules; file count on disk re-checked after untracking
Claude Code	Writing <code>src/config.py</code> : path objects, controlled vocabularies, dish-name character rules	Executed a check resolving every declared path against the filesystem, asserting each OpenRefine export's column shape, confirming no directory is created as a side effect, and spot-checking the log-tag and price formats
Claude Code	Writing <code>src/cleaning_utils.py</code> : the pure value-level cleaners each per-table module calls	Tested against real values taken from the profiling rather than invented ones; one fabrication bug found and fixed (a dish named <code>&</code> cleaned to <code>AND</code>). Re-run over all 394,297 dish names: no result outside the target charset, no empty results

Claude Code	Writing <code>src/provenance.py</code> : per-row log accumulation, cell-level change capture, stage snapshots, and the <code>cleaning_log.csv</code> rollout	Ran four real stages over <code>Menu_OR.csv</code> (5,236 rows) and checked the accumulated log, the change log contents, the column ordering, and that run directories appear only when a stage writes. Re-ran the sequence from scratch and compared SHA-256 hashes to confirm byte-identical output
Claude Code	Writing the four per-table cleaning modules and <code>src/run_pipeline.py</code> , each annotated for YesWorkflow	Each module run over its full export and its row counts checked against the input; one defect caught per module (see section A.1.6)
Claude Code	The price outlier rule: winsorizing the right tail, with the limit chosen by measuring candidates against the U1 metric	Thresholds confirmed against the maxima in the load-ready CSVs; row counts unchanged either side of the stage; 4,710 capped cells present in the change logs with before and after values
Claude Code	Clean-room rebuild: deleting all 37 generated files and 18 directories, then regenerating from the eight inputs	All 37 files reproduced byte-for-byte, verified by SHA-256 against a fingerprint taken before deletion

Each task below records the instruction given, what came back, and how we checked it.

A.1.1 Profiling the raw data

These are the results of my OpenRefine profiling. Use them as the objectives for the Python/pandas cleaning.

Menu: id has no blanks. date has 586 blanks; ditch those rows, since a menu with no date cannot be grouped into a time period. There are extreme years (1091, 0190 and 2928) which are not workable data, so abandon those rows too. currency and currency_symbol line up, with the blanks on the same rows, so we do not need the symbol column. There are 39 currency variants and a fair number of blanks, and I want to try to rescue some of those rows: possibly by converting the source currency to USD against the menu's own time period, or by clustering location and using an LLM to infer the country with a 0–1 confidence score. status has only 167 "under review" rows against complete; throw those away.

MenuPage: the data is solid; id and menu_id are populated on every row. All we use is those two columns. Verify they are numeric, and leave foreign-key dependencies to SQL.

MenuItem: id and menu_page_id have no blanks and look numeric. Around 446k rows have no price, and high_price is blank whenever price is, so there is no within-row substitute when price is missing; we need to see whether those rows can be rescued another way. dish_id is populated on every row.

Dish: the good news is that most rows have a lowest and highest price, but only 171,731 of 423,397 have both. That may be a problem to solve at the SQL stage; for now this is as good as we can do.

These objectives became rules 1 through 8 in section 1.1.C. Two of them were investigated and dropped rather than implemented. The currency rescue (Section A.1.11 below) and, later, the symmetric outlier trim (Section A.1.9 below). Both rejections are recorded with the evidence that produced them.

Before writing code against the figures above, the tool checked them against the raw files and reported three that did not hold: `currency` distinct values (43, not 39), `status = under review` (174, not 167), and `MenuItem.dish_id` blanks (241, not none). The `dish_id` figure was independently corroborated by our Phase-I dataset description, which had already recorded empty `dish_id` values. So the correction stands, and the "no blanks" reading above was a facet we had misread.

A.1.2 Reorganizing the data tree

Given my OpenRefine exports, reorganize data/ so each source CSV has its own raw/interim/final tree, with one directory per cleaning function inside interim for provenance, and move the raw and OpenRefine files into it.

It first created the function directories up front. We rejected that: pre-created directories have to be kept in sync with the code by hand, which makes adding a cleaning function a two-place change. We had it move directory creation into the code so each function creates its own run directory on first execution. It agreed and additionally dropped the numbering it had applied to those directories, on the grounds that numbered directories have to be renumbered whenever a stage is inserted, which is the same friction we were removing.

Ignore the whole data/ tree in git. The datasets go out over a shared drive instead.

Before making the change it reported that this was not just a preference: `MenuItem.csv` is 113 MiB, above GitHub's 100 MiB per-file limit, so the push would have been rejected outright. It also noted that the course instructions require the datasets to be shared via Box at submission time, which is separate from whatever we use during development.

The move script ran preflight assertions (every source present, no destination pre-existing) and compared byte sizes before and after each move; we confirmed the new ignore rules with git check-ignore and re-counted the files on disk after untracking.

A.1.3 Pipeline configuration module

Write the pipeline config module: path objects per table, a Dollars -> USD currency map, the date-part columns, the dish-name character rules (uppercase ASCII alphanumerics plus apostrophe and comma; separators become spaces; & becomes AND; accents folded;

non-Latin names marked UNKNOWN), and a per-row log column convention tagged <column>:<operation> so change counts can be reported per column.

Produced `src/config.py`. Two additions it argued for and we kept: an assertion of each OpenRefine export's expected column shape, so that re-exporting with a renamed column fails immediately rather than deep inside a cleaning function; and a `log_tag()` helper enforcing the `<column>:<operation>` format, which makes the per-column change counts in section 2.1.B computable.

It proposed a `MIN_PRICED_ITEMS_PER_MENU` threshold, which we removed as premature.

Verified by executing a check that resolved every declared path against the filesystem, asserted each export's column shape, confirmed no directory is created merely by referencing its path, and spot-checked the log-tag and price formats.

A.1.4 Value-level cleaning functions

Write the cleaning utilities the pure value-level functions the per-table cleaners will call.

Produced `src/cleaning_utils.py`: `normalize_text`, `clean_int`, `clean_decimal_2dp`, `split_iso_date`, `map_currency`, and `clean_dish_name`. Each returns the cleaned value together with the list of operations that produced it, so the per-row `log` column is a byproduct of cleaning rather than something reconstructed afterwards. It proposed the rule that an operation is recorded only when it actually changed the value, on the grounds that a type coercion applied uniformly would otherwise tag all 1.3M `MenuItem` rows identically and make the log useless.

We had it test the functions against real values drawn from the profiling rather than invented ones. That run failed on one case: a dish named "&" was being cleaned to "AND", because the ampersand-to-word expansion ran before the check for symbol-only names. It was corrected to detect symbol-only names before any substitution.

Checked over the full column: 394,297 names processed, no result outside the target character set, no empty result, 647 names marked `UNKNOWN`, and distinct names falling from 390,201 to 349,480.

A.1.5 Provenance module

Write the provenance layer the per-table cleaners sit on: it should apply the value-level cleaners to columns, accumulate the per-row log, capture cell-level before/after changes, and write each stage's output directory. Row-level change logs for semantic transforms only. Formatting-only work records counts instead. The cleaning functions create their own run directories, so directory creation belongs here.

Produced `src/provenance.py`. Each cleaning stage constructs a `StepRecorder`, which writes three artifacts into `data/<table>/interim/func-<step>/`: the stage snapshot, a cell-level change log, and a summary of counts per column and operation. The summaries are concatenated into `data/reports/cleaning_log.csv`, which is the source for the change table in section 2.1.B.

Two things it raised that we adopted. It moved the OpenRefine column-shape assertion into the loader, so re-exporting from OpenRefine with a renamed column fails on load with a message naming the file and the expected columns, rather than surfacing later as a confusing error inside a cleaning function. And it wrote all CSVs with LF line endings and no timestamps, on the grounds that our checklist requires the workflow be rerunnable.

Verified by running four real stages over `Menu_OR.csv` (5,236 rows): identifiers coerced, date split into year/month/day with the original `date` retained, currency mapped, `status` dropped. We confirmed the accumulated log reads `date:date_split;currency:currency_mapped;status:column_dropped`, that no run directory exists until a stage writes one, that opting out of change recording suppresses the change file, and that the change log records `Dollars -> USD` for all 5,236 rows. We then re-ran the whole sequence from scratch and compared SHA-256 hashes to confirm the output was byte-identical.

The identifier stage produced an empty summary, because the identifiers in the preliminary OpenRefine export were already clean integers. We kept the stage rather than removing it: a measured no-op is evidence, and section 2.1.B should report it as zero rather than omit it.

A.1.6 Per-table cleaning modules

Write one cleaning module per source table, each calling the value-level cleaners through the provenance layer. Annotate each stage for YesWorkflow: inputs and outputs so the graph can be generated later, configuration constants marked as parameters, and the output filenames named in the stage description so a node in the graph points at the file it produced. Not every table runs the same stages; the graph should show that.

Written one at a time and reviewed before moving to the next.

`src/clean_menu_page.py` is a single stage identifier coercion. `MenuPage` carries only `id` and `menu_id`, so it has no dates, prices, text, or categoricals to clean, against `Menu`'s four stages. Run over the export: 66,937 rows in, 66,937 out, none dropped. The stage summary is empty because the preliminary `OpenRefine` export already held clean integers; we kept the stage rather than deleting it, so section 2.1.B can report a measured zero instead of omitting the column.

It declined to resolve dangling `menu_id` values in this module, on the grounds that inclusion dependencies need a cross-table join and are Madalyn's SQL IC/FD/IND violation runs; the module reports the distinct `menu_id` count (19,816) so the two results can be reconciled rather than double-counted. It also chose to flag unparseable identifiers rather than raise, since raising would stop the run and hide the offending row, where our convention is to flag and carry every row forward.

`src/clean_menu.py` contains four stages: identifiers, date split, currency mapping, status drop. 5,236 rows in and out. Observed year range 1851–2012, no unusable years, no identifiers that failed to parse, and no currency outside our mapping.

It added a guard we had not asked for: the status column is dropped only after confirming it is constant. The justification for dropping it is that the preliminary `OpenRefine` filter left only `complete` rows, so if a later re-export widened that filter, dropping the column would destroy the distinction with no trace. The module keeps the column and warns instead.

`src/clean_dish.py` has four stages: identifiers, name normalization, price decimals, outlier capping. 394,297 rows in and out, in 10.7 seconds. Distinct names fell from 390,201 to 349,480, so 40,721 name variants collapsed. 647 names were marked `UNKNOWN`. Prices confirmed the profiling exactly: 222,566 zero or negative `lowest_price` values, and 171,731 dishes (43.6%) usable as a price fallback with both prices positive.

The first run produced a 42 MiB change log, of which 380,613 of 383,289 rows recorded nothing but a change in capitalization. We had it narrow the change log to rows where something beyond uppercasing happened, which cut the file to 16 MiB while leaving every count in the summary intact: the accent folds, ampersand expansions and `UNKNOWN` marks are now readable as evidence instead of buried.

`src/clean_menu_item.py` contains four stages: identifiers, price decimals, outlier capping, omit check. 1,332,668 rows in, 1,332,432 out, 236 omitted and written to a separate file with the reason recorded on each row. This is the only table the pipeline removes rows from.

We defined the omit rule as no `dish_id` and no positive price, since either one alone still leaves a route to a price. The module reports the components separately so the narrowness is visible: 239 rows have no `dish_id`, 446,210 have no positive price, and only 236 have both. The 445,974 rows with a dish but no price are explicitly kept for the pricing fallback join, which happens in SQL where Dish is available.

Checking the output caught a defect that would have reached Madalyn: `dish_id` was being written as `1.0` rather than `1`. Because 239 rows have no `dish_id`, pandas had upcast the whole column to floating point; `id` and `menu_page_id` escaped only because they contain no nulls. Written that way, `MenuItem.dish_id` would not have matched `Dish.id` without a cast, breaking the price fallback join. Fixed by using pandas' nullable integer type, and applied to every identifier column in all four modules rather than only the one that failed.

A.1.7 Output layout correction

The load-ready CSVs should not carry the provenance columns. SQLite has no use for `source_file`, `source_row_num`, or `log`. Put the load-ready output in a cleaned directory under `interim`, and remove the per-table final directories: `data/final/` belongs to the SQL workflow. Also drop the ISO date string from Menu now that year, month, and day exist.

Reviewing the first complete run showed the load-ready CSVs carrying three provenance columns that no U1 query refers to. Stripping now happens at the final write only, so every upstream stage snapshot still carries the full audit trail. The Menu date string is dropped through the same logged mechanism as any other column removal, and the original remains visible one stage upstream in `Menu_after_id-numeric.csv`, which keeps the derivation checkable.

Load-ready output after the correction:

File	Columns
Menu_cleaned.csv	id, currency, year, month, day
MenuPage_cleaned.csv	id, menu_id
Dish_cleaned.csv	id, name, lowest_price, highest_price
MenuItem_cleaned.csv	id, menu_page_id, price, high_price, dish_id

A.1.8 YesWorkflow model for W2

Annotate the pipeline for YesWorkflow: inputs, outputs, and configuration constants as parameters, with output filenames named in each stage description. The graph should show that not every table runs the same sequence.

Annotations were added to each cleaning module as it was written rather than retrofitted, so the model derives from the code that runs. Because the YesWorkflow tool was not installed at the time, it also wrote a linter for the annotations, which checks that blocks are balanced, that every declared input has a producing output, that every parameter names a real constant in `src/config.py`, and that every file reference exists. That linter caught a defect before the tool was available: all four chains ended in an output nothing consumed, because the final write step had no block of its own, so each chain would have rendered disconnected from its own output.

Installing YesWorkflow (0.2.1.2) then exposed two further problems that the tool reports no error for:

1. Without `-c "extract.comment=#"`, YesWorkflow reads Python docstrings as the comment syntax and finds no annotations at all, exiting successfully having produced nothing.
2. Argument order is significant. YesWorkflow concatenates its source arguments into a single stream, so a block opened in the first file is closed from the last. With the outer block opened inside `run_pipeline.py` it closed before the cleaning modules were read, and the tool produced a 21-block model rendered as a 6-node graph containing none of the cleaning chains. The outer frame was moved to `workflow/W2_frame.yw`, which is passed first.

A third mistake was self-inflicted and worth recording because it is easy to repeat: a comment explaining the annotation structure mentioned the `begin` keyword in an ordinary sentence, and YesWorkflow parsed that sentence as a real annotation, opening a duplicate block. Its matching is loose enough that the keywords cannot appear in prose anywhere in the repository.

Final model: 21 nodes and 24 edges at the top level, with per-table detail graphs of 16, 16, 17, and 10 nodes. Verified by checking that each generated file is well-formed DOT with balanced braces, that the four table chains appear as sub-workflow nodes, and that each per-table graph contains its own stages.

A.1.9 Price outlier rule

Add the extreme-value check to the cleaning pipeline. this is more awkward to do in SQL. Winsorize the values, keep the percentiles in config.py, and remove 3% of outliers from each tail.

Asked to implement a symmetric 3% trim with row removal, it measured the effect on the U1 metric first and reported that the rule would break it. Trimming both tails moved the average price per menu by +8.7% for the cheapest quartile of menus and -32.0% for the priciest. The cause is that within dollar-denominated menus the median item is \$0.40, so the 97th percentile is only \$4.00: trimming above it removes most of what makes an expensive menu expensive, while the bottom trim deletes legitimate cheap items.

It also corrected an earlier claim of its own. It had previously described a `MenuItem.price` of 180,000 as a suspicious outlier; checking the menu behind it showed an Italian Lire menu at the Grand Hotel Baglioni, where 180,000 is an ordinary price. Of the 1,183 items priced above 1,000, only 4 are dollar-denominated, the rest are Lire, Francs, forint and Yen on menus our currency filter already excludes.

Based on those results we changed the rule to winsorization of the right tail only, which means capping at a percentile rather than deleting, and leaving the left tail alone. The limit was then chosen by measuring candidates against the same tier test rather than by convention:

Limit	Threshold	In-scope capped	cheapest	Q2	Q3	priciest
5.0%	\$9	9,482	-0.2%	-0.9%	-3.3%	-13.9%
1.0%	\$110	205	0.0%	0.0%	0.0%	-0.6%
0.5%	\$300	21	0.0%	0.0%	0.0%	-0.1%
0.1%	\$1,500	2	0.0%	0.0%	0.0%	-0.0%

0.5% caps 4,251 `MenuItem.price` values of which only 28 sit on dollar menus, so it targets the currency artifacts almost exclusively while moving the priciest in-scope quartile by 0.1%. The limit and the affected columns live in `src/config.py` with this evidence recorded beside them.

Verified by confirming the maxima in the load-ready CSVs match the thresholds, that the row count is unchanged either side of the stage, that 4,710 capped cells appear in the change logs with their before and after values, and that the capped rows carry a `winsorized` tag in the `log` column.

A.1.10 Clean-room rebuild of the whole data set

Remove every interim table we generated, leaving only the raw and OpenRefine CSVs. Run the pipeline end to end to regenerate them, regenerate the YesWorkflow graphs, and document each step.

The goal of this run was to test a claim we had been making rather than to produce anything new. Up to here the generated tree had accumulated across many sessions, so "the pipeline reproduces the data" had never actually been demonstrated from nothing.

We used the following sequence:

1. Fingerprinted the existing output, so the rebuild could be compared against it rather than merely inspected.
2. Deleted every generated artifact (37 files and 18 directories), keeping only the eight inputs: four raw CSVs and four OpenRefine exports. A preflight listed exactly what would be removed and confirmed all eight inputs were on the keep side before anything was deleted.
3. Ran `python -m src.run_pipeline` from the empty tree. 32.9 seconds. Every stage directory was created by the code as it ran; none had to exist beforehand.
4. Compared the rebuilt tree against the fingerprint. All 37 files reproduced identically.
5. Regenerated the workflow model from scratch. Deleted the ten existing graph artifacts and regenerated `w2.gv` plus the four per-table graphs and their PNG renders.

Row counts and every measured figure came out the same as the values reported in section 2.1.B, which is the result we wanted: the numbers in this report describe a data set that can be rebuilt from its inputs, not one that exists only because of the order we happened to do things in.

A.1.11 What we decided, what changed our plan, and how we checked it

This subsection closes out section A.1 as a whole rather than describing a single task.

Decisions made by the team

The preliminary OpenRefine filtering that produced the `*_OR.csv` exports was Charlene's work, done in the OpenRefine UI; our AI tool never touched it. The definitive OpenRefine profiling and cleaning is Charlene's and is outside the scope of this appendix. Also ours: the directory scheme, the decision to keep the data out of git, the rule that cleaning functions create their own run directories. Additionally, how far to take

the UNKNOWN rule for non-ASCII dish names, whether punctuation should be deleted or converted to word boundaries, and whether the row-level change log should cover formatting-only operations were also part of our decision points.

Findings that changed our plan

Two findings, both recorded in Plan vs. actual:

1. Blank `Menu.currency` marks a menu that was never price-transcribed. We had planned to recover those 11,089 menus (tripling our sample) by inferring the country from `location` with an LLM and deriving the currency. Profiling showed the 11,089 blank-currency menus carry 361,600 transcribed items with 473 prices between them, against 795,388 prices on the 5,549 dollar-denominated menus. Currency is blank because there was nothing to denominate, so a correct currency label would have recovered no usable menu. We dropped the idea. It also showed the fallback our Phase-I plan relied on is narrower than assumed: about 81% of items lacking a price sit on menus that were never price-transcribed at all.
2. A column identity error in our own notes. We had been treating `Menu.location` as the geographic field; it holds the venue name (`Hotel Eastman`), while `Menu.place` holds the geography (`HOT SPRINGS, AR`).

We did not copy any reported count directly from the tool without checking it. Every figure that reached a cleaning rule was reproduced either by the preliminary OpenRefine facets or by a second, independently written check. The blank-currency finding above rests on a single pandas join, so it is re-run in SQL once staging is loaded (see Section 2.2).

A.2 Other tools

Tools used beyond the course toolset of RegEx, OpenRefine, Datalog/Logica, SQL, and Python. The instructions require these to be documented.

Tool	Used for	Notes
YesWorkflow	Generating the W2 inner workflow model from annotations embedded in the pipeline source	Named in the project instructions as an option for the workflow model. Not a Python package. A Java jar, run as <code>java -jar</code> . Requires <code>-c "extract.comment=#"</code> , without which it silently finds no annotations in Python source
Graphviz (dot)	Rendering the generated <code>.gv</code> files to PDF	Only needed for the render; the <code>.gv</code> files themselves are produced by YesWorkflow. Its Windows installer

		does not add itself to PATH when installed non-interactively
pandas	The entire Python/Pandas cleaning pipeline mentioned above	Within the course's Python toolset; noted for completeness since all type coercion, normalization, and provenance accounting run through it
pypdf	Reading the course instructions PDF while planning	Development-time only. Deliberately not in <code>requirements.txt</code> . The pipeline does not import it
Git / GitHub	Version control for code and documentation	The datasets are excluded: <code>MenuItem.csv</code> is 113 MiB, above GitHub's 100 MiB per-file limit, and the cleaned outputs are regenerated on every run. Raw and cleaned data are shared out of band and, at submission, via Box per the instructions

Appendix B - YesWorkflow Graphs

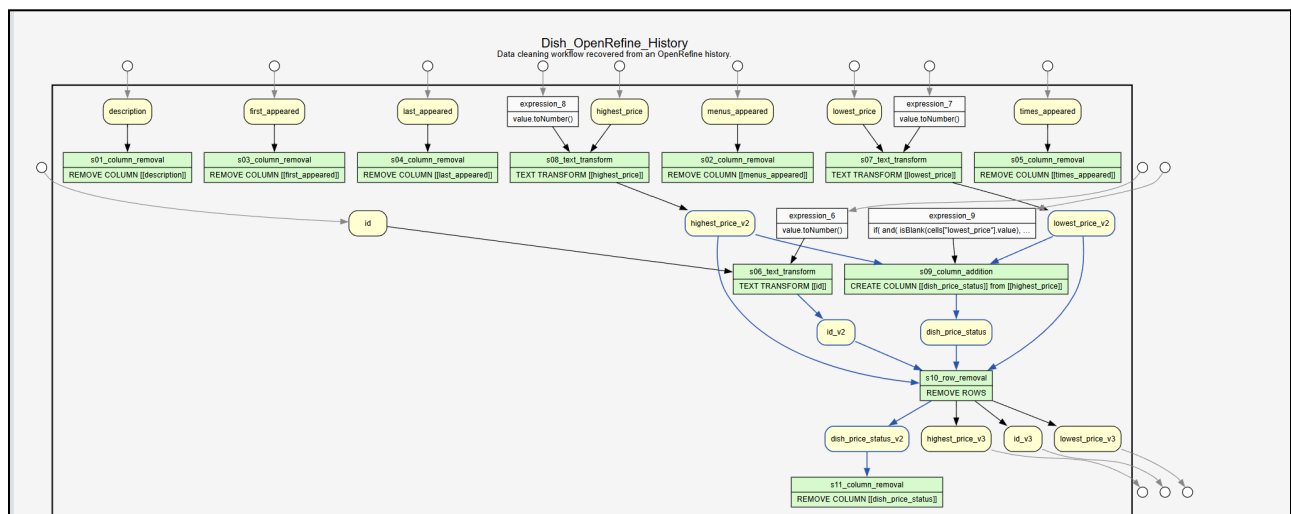


Figure B.OR.1: Shows an overall view of Yes workflow graph based on `Dish_OpenRefine_History.json`

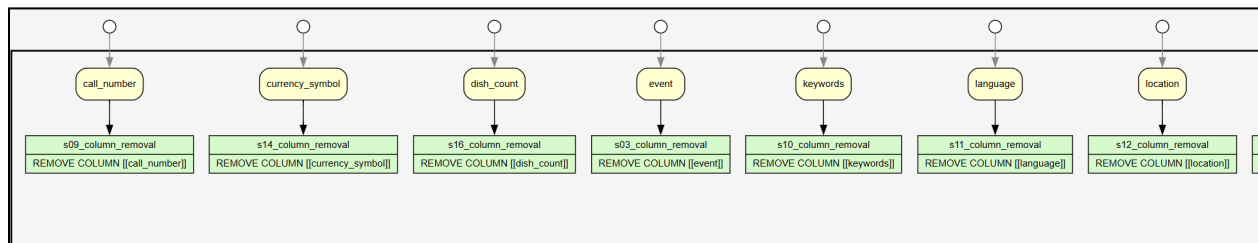


Figure B.OR.2: Shows a zoomed in portion of Yes workflow graph based on Menu_OpenRefine_History.json

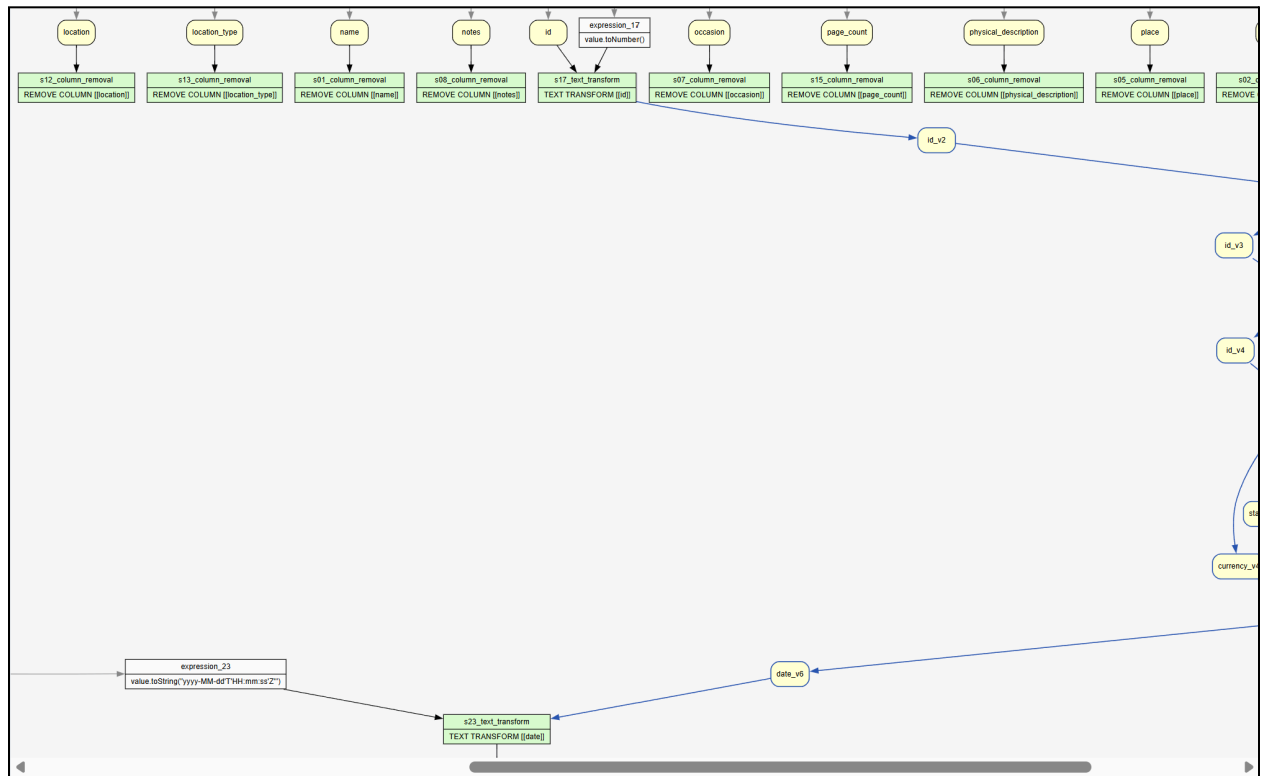


Figure B.OR.3: Shows a zoomed in portion of Yes workflow graph based on Menu_OpenRefine_History.json

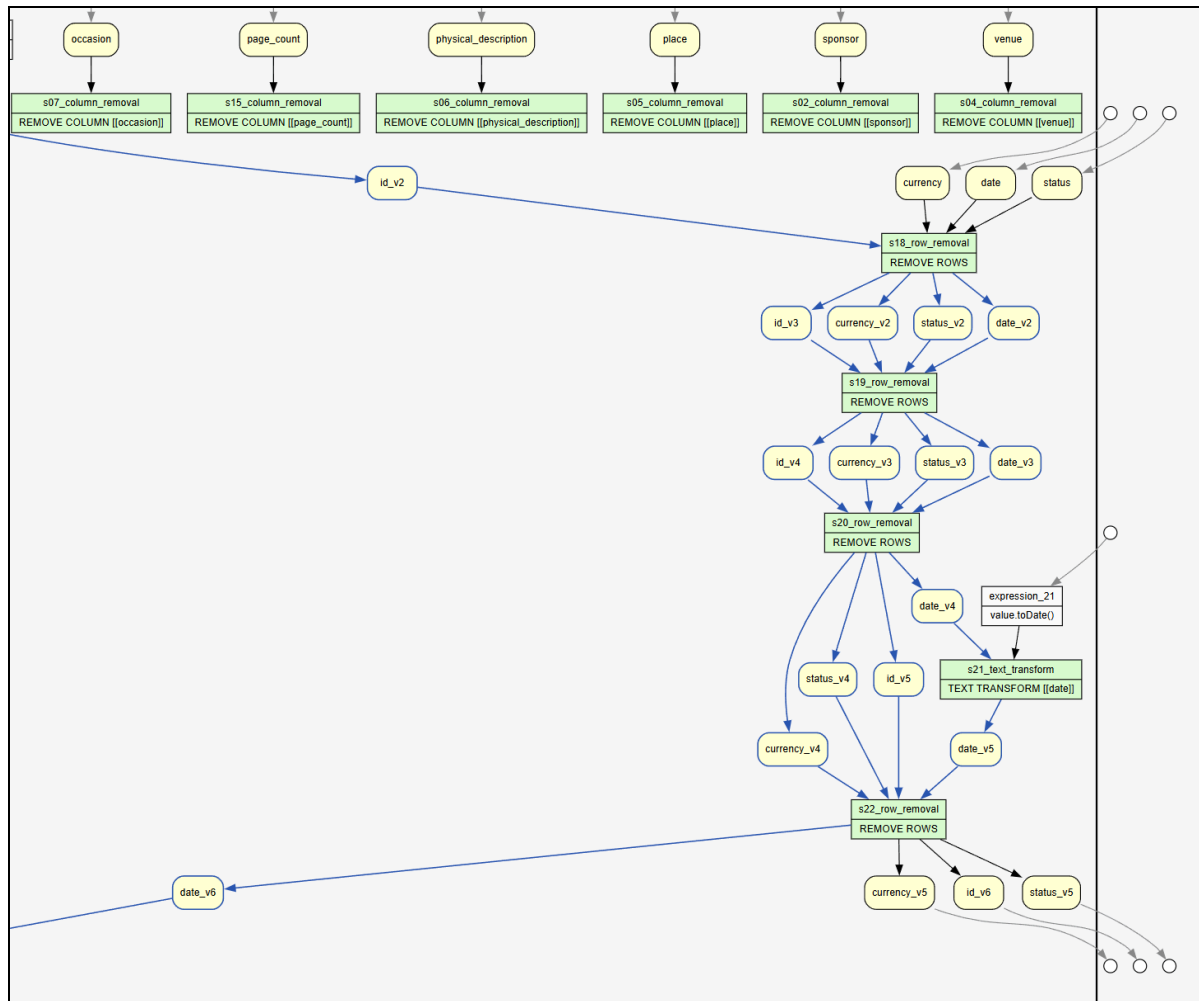


Figure B.OR.4: Shows a zoomed in portion of Yes workflow graph based on Menu_OpenRefine_History.json

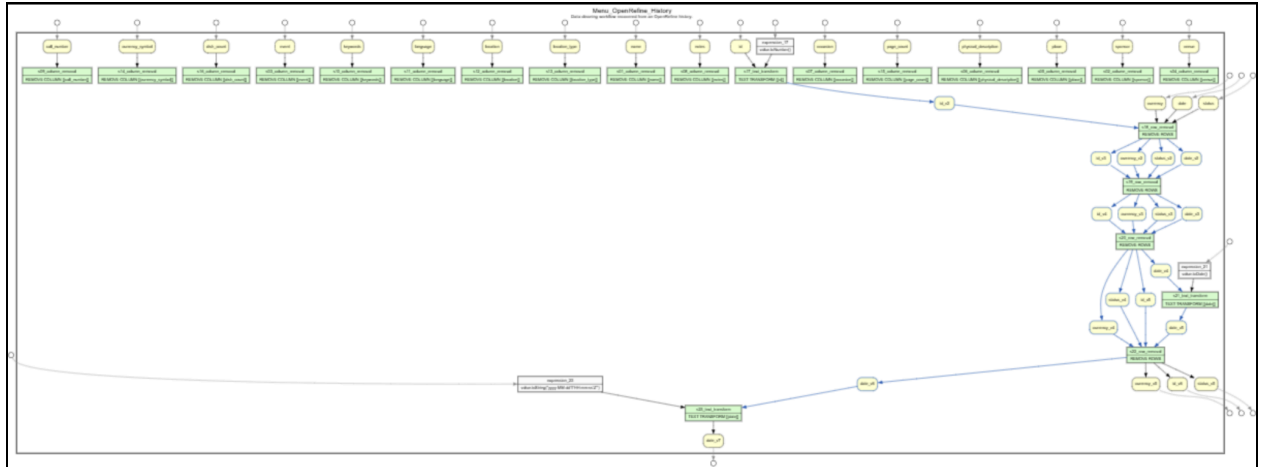


Figure B.OR.5: Shows an overall view of Yes workflow graph based on Menu_OpenRefine_History.json

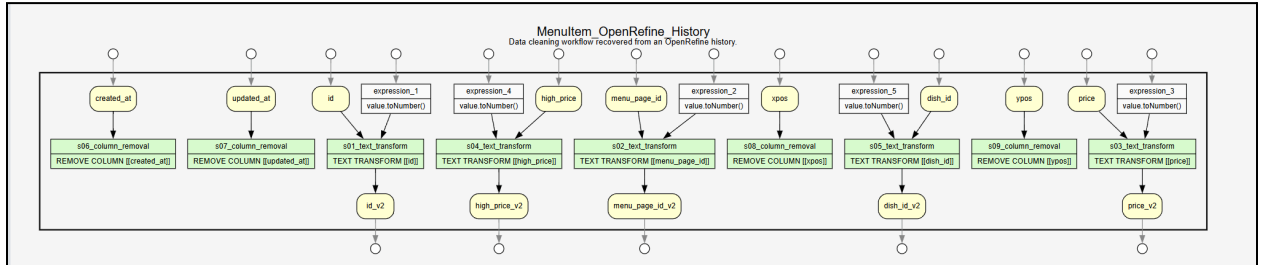


Figure B.OR.6: Shows an overall view of Yes workflow graph based on MenuItem_OpenRefine_History.json

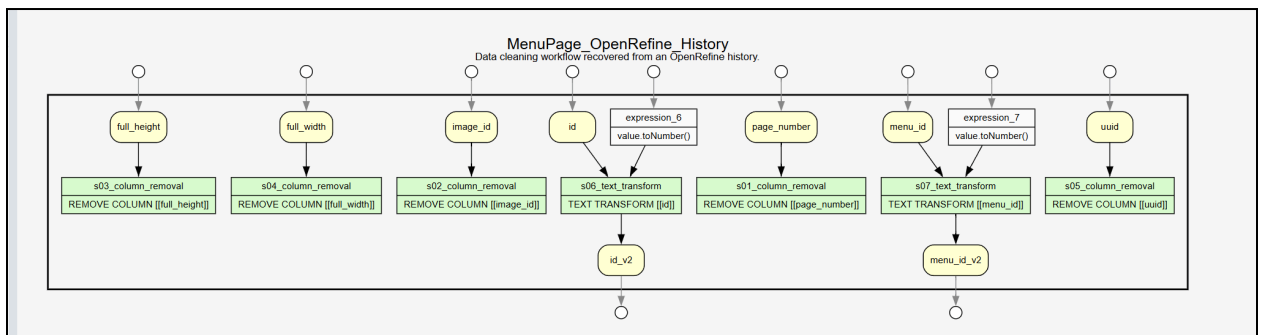


Figure B.OR.7: Shows an overall view of Yes workflow graph based on MenuPage_OpenRefine_History.json

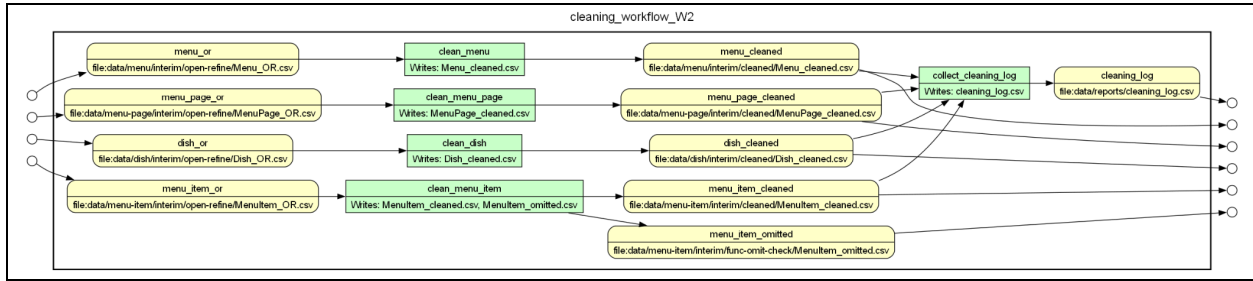


Figure B.PP.1: Top-level graph representing the high-level cleaning flows for each of our input files (outputs from OpenRefine)

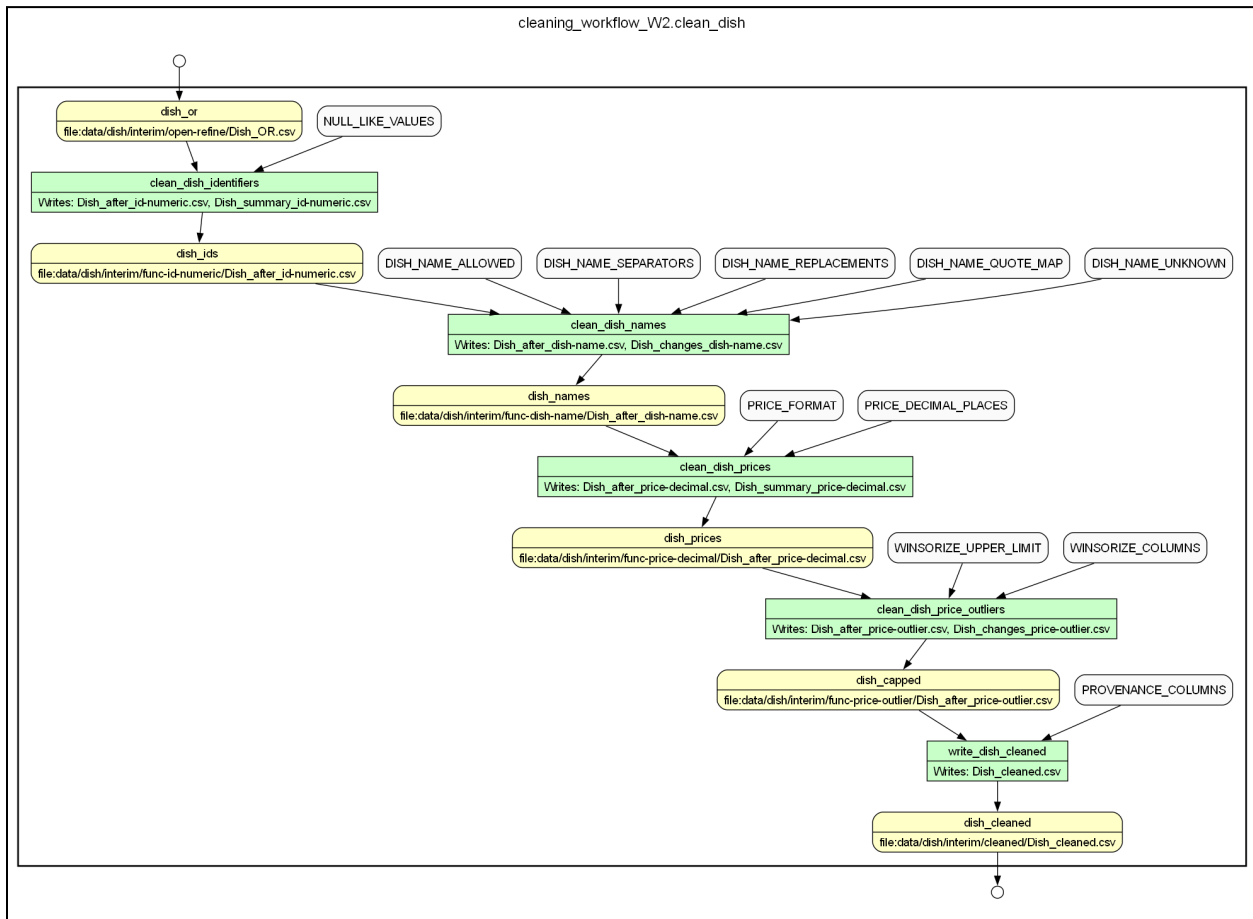


Figure B.PP.2: File-level workflow graph for the Dish data file

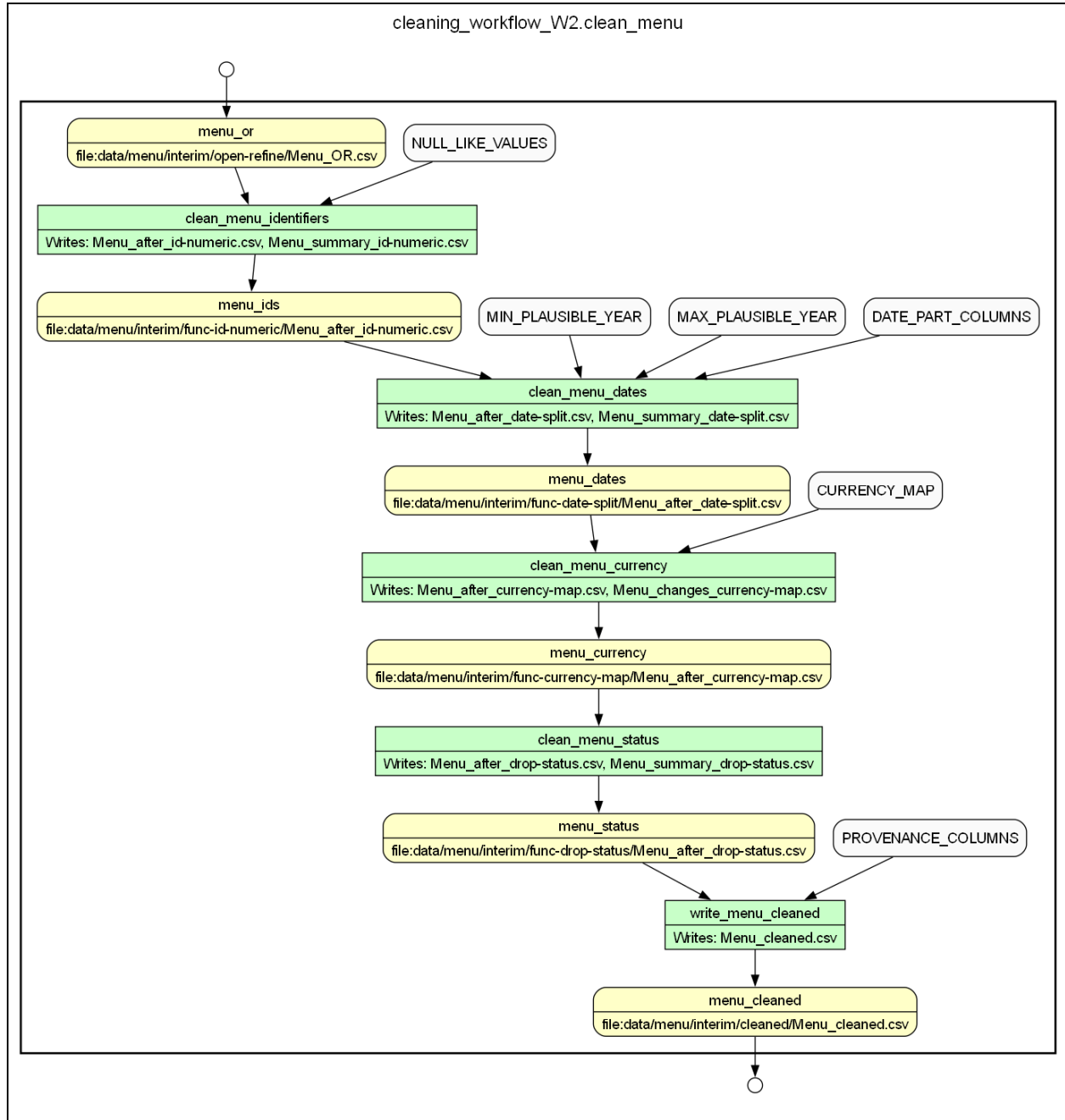


Figure B.PP.3: File-level workflow graph for the Menu data file

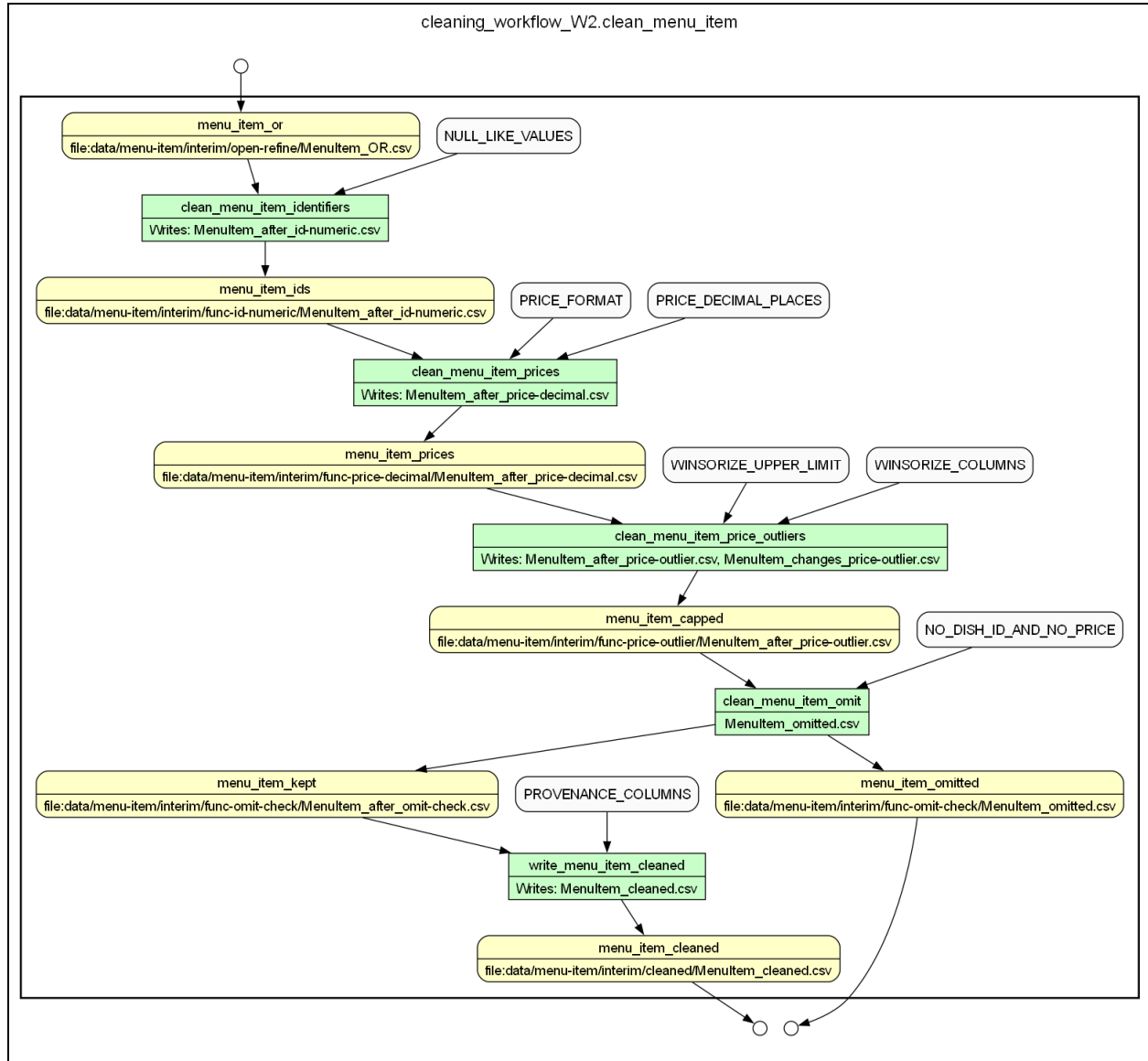


Figure B.PP.4: File-level workflow graph for the Menuitem data file

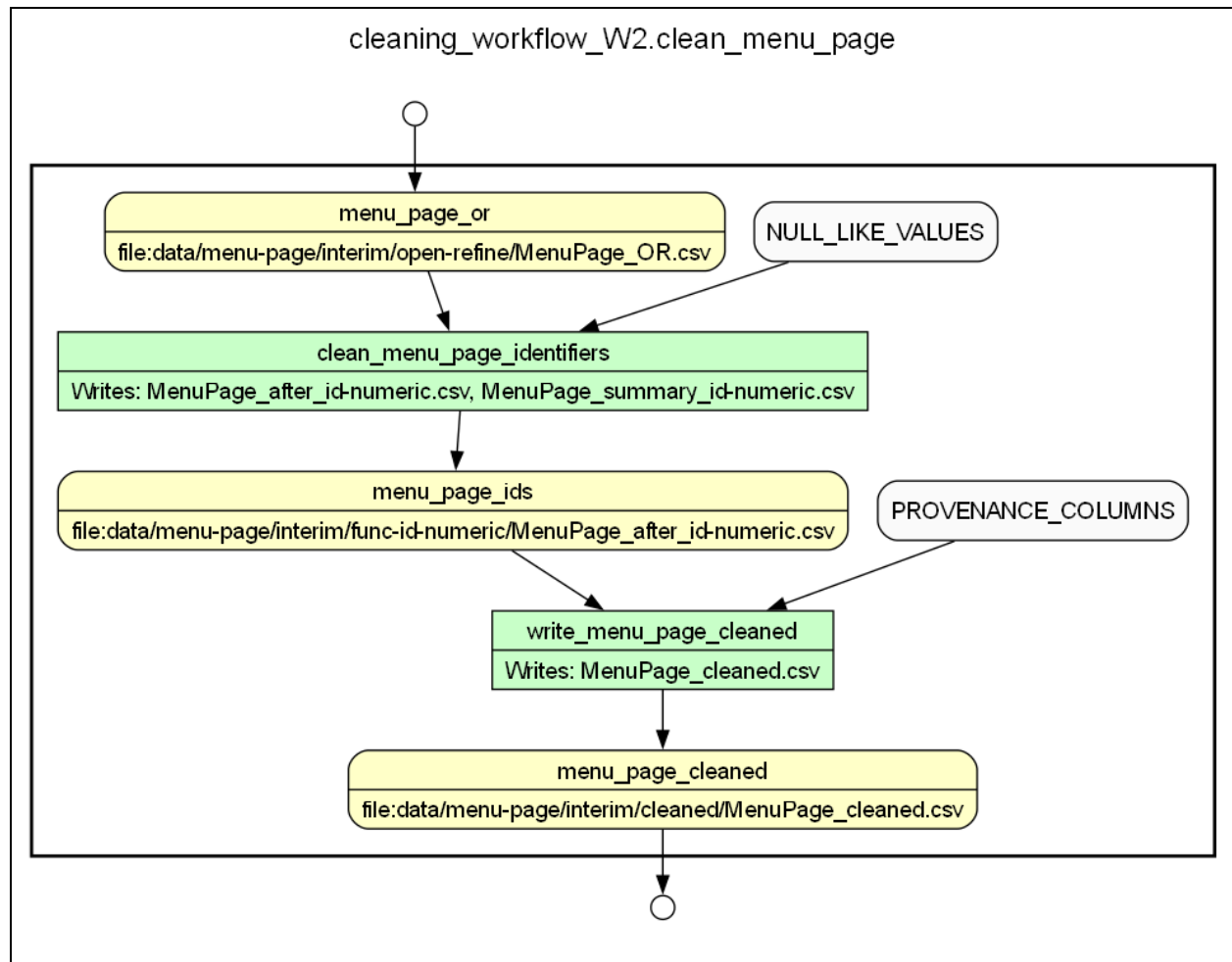


Figure B.PP.5: File-level workflow graph for the MenuPage data file